The Concise TypeScript Book

Simone Poggiali

Краткая книга по TypeScript

Краткая книга по TypeScript представляет собой исчерпывающий и лаконичный обзор возможностей TypeScript. Она содержит понятные объяснения всех аспектов последней версии языка — от мощной системы типов до расширенных возможностей.

Независимо от того, являетесь ли вы начинающим или опытным разработчиком, эта книга станет ценным ресурсом для углубления ваших знаний и совершенствования навыков работы с TypeScript.

Эта книга полностью бесплатна и распространяется с открытым исходным кодом.

Я считаю, что качественное техническое образование должно быть доступно каждому. Поэтому я предоставляю книгу бесплатно и регулярно обновляю её, внося улучшения и добавляя новые примеры.

Познакомьтесь с **расширенным изданием «Краткой книги по TypeScript»**.



Для читателей, которые хотят выйти за рамки издания с открытым исходным кодом, **расширенное издание «Краткой книги по TypeScript: React и практические шаблоны для TypeScript 7»** содержит дополнительные эксклюзивные материалы, посвящённые практическому применению.

Расширенное издание включает:

- **Обновлено для TypeScript 7** — рассмотрены новейшие возможности TypeScript 7 и усовершенствования языка.
- **TypeScript и React** — практические рекомендации по типизации компонентов, пропсов, хуков, событий,

дочерних элементов, рефов и распространённых шаблонов React.
- **Рецепты TypeScript для реальных проектов** — наглядные примеры решения практических задач, с которыми разработчики сталкиваются при создании и сопровождении приложений на TypeScript.

Приобретая расширенное издание, вы также напрямую поддерживаете дальнейшую разработку и сопровождение бесплатной книги с открытым исходным кодом.

Расширенное издание на английском и итальянском языках доступно на Amazon по всему миру. Узнайте больше о расширенном издании и купите его на Amazon.

Поддержать проект

Если бесплатная книга помогла вам исправить ошибку, разобраться в сложной концепции или продвинуться в карьере, пожалуйста, рассмотрите возможность поддержать мою работу, внеся любую сумму (рекомендуемый взнос — **5 долларов**) или угостив меня кофе.

Ваша поддержка помогает мне поддерживать актуальность материалов и дополнять их новыми примерами, более понятными объяснениями и практическими рекомендациями.





Переводы

Эта книга переведена на несколько языков, включая:

Английский

Болгарский

Немецкий

Французский

Индонезийский

Итальянский

Японский

Корейский

Польский

Португальский (Бразилия)

Шведский

Турецкий

Вьетнамский

Китайский

Испанский

Тайский

Русский

Арабский Чешский

Загрузки и веб-сайт

Вы также можете загрузить версию в формате EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Онлайн-версия доступна по адресу:

https://gibbok.github.io/typescript-book

Содержание

- Краткая книга по TypeScript
  - Поддержать проект
  - Переводы
  - Загрузки и веб-сайт
  - Содержание
  - Введение
  - Об авторе
  - Введение в TypeScript
    - Что такое TypeScript?
    - Зачем нужен TypeScript?
    - TypeScript и JavaScript
    - Генерация кода с помощью TypeScript
    - Современный JavaScript уже сейчас (понижение версии)
  - Начало работы с TypeScript
    - Установка
    - Настройка

        - Файл конфигурации TypeScript
            - target
            - lib
            - strict
            - module
            - moduleResolution
            - esModuleInterop
            - jsx
            - skipLibCheck
            - files
            - include
            - exclude
        - importHelpers
        - Рекомендации по переходу на TypeScript
    - Изучение системы типов
        - Языковая служба TypeScript
        - Структурная типизация
        - Основные правила сравнения в TypeScript
        - Типы как множества
        - Задание типа: объявления типов и утверждения типов
            - Объявление типа
            - Утверждение типа
            - Внешние объявления
        - Проверка свойств и проверка избыточных свойств
        - Слабые типы
        - Строгая проверка объектных литералов (свежесть)
        - Вывод типов
        - Более сложные случаи вывода типов
        - Расширение типов

        - Const
            - Модификатор const для параметров типов
            - Утверждение const
        - Явная аннотация типа
        - Сужение типов
            - Условия
            - Выбрасывание исключения или возврат
            - Дискриминируемое объединение
            - Пользовательские защитники типа
            - Сужение с помощью switch (true)
    - Примитивные типы
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null и undefined
        - Массив
        - any
    - Аннотации типов
    - Необязательные свойства
    - Свойства только для чтения
    - Индексные сигнатуры
    - Расширение типов
    - Литеральные типы
    - Вывод литеральных типов
    - strictNullChecks
    - Перечисления
        - Числовые перечисления
        - Строковые перечисления
        - Константные перечисления
        - Обратное сопоставление

        - Внешние перечисления
        - Вычисляемые и константные члены
    - Сужение типов
        - Защитники типа typeof
        - Сужение по истинности
        - Сужение по равенству
        - Сужение с помощью оператора in
        - Сужение с помощью instanceof
    - Присваивания
    - Анализ потока управления
    - Предикаты типов
    - Дискриминируемые объединения
    - Тип never
    - Проверка исчерпываемости
    - Объектные типы
    - Тип кортежа (анонимный)
    - Именованный тип кортежа (с метками)
    - Кортеж фиксированной длины
    - Тип-объединение
    - Типы-пересечения
    - Индексирование типов
    - Тип из значения
    - Тип из возвращаемого значения функции
    - Тип из модуля
    - Сопоставляемые типы
    - Модификаторы сопоставляемых типов
    - Условные типы
    - Дистрибутивные условные типы
    - Вывод типа с помощью infer в условных типах
    - Предопределённые условные типы
    - Шаблонные типы объединений
    - Тип any

- Тип unknown
- Тип void
- Тип данных never
- Интерфейс и тип
  - Общий синтаксис
  - Базовые типы
  - Объекты и интерфейсы
  - Типы объединения и пересечения
- Встроенные примитивные типы
- Распространённые встроенные объекты JS
- Перегрузки
- Слияние и расширение
- Различия между типом и интерфейсом
- Класс
  - Общий синтаксис класса
  - Конструктор
  - Закрытые и защищённые конструкторы
  - Модификаторы доступа
  - Геттеры и сеттеры
  - Автоаксессоры в классах
  - this
  - Свойства-параметры
  - Абстрактные классы
  - С обобщёнными типами
  - Декораторы
    - Декораторы классов
    - Декоратор свойства
    - Декоратор метода
    - Декораторы геттеров и сеттеров
    - Метаданные декораторов
  - Наследование
  - Статические члены

        - Инициализация свойств
        - Перегрузка методов
    - Обобщения
        - Обобщённый тип
        - Обобщённые классы
        - Ограничения обобщений
        - Контекстное сужение обобщённых типов
    - Стираемые структурные типы
    - Пространства имён
    - Символы
    - Директивы с тройной косой чертой
    - Манипуляции с типами
        - Создание типов из типов
        - Типы индексированного доступа
        - Служебные типы
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>

        - Uppercase<T>
        - Lowercase<T>
        - Capitalize<T>
        - Uncapitalize<T>
        - NoInfer<T>
    - Прочее
        - Ошибки и обработка исключений
        - Классы-примеси
        - Асинхронные возможности языка
        - Итераторы и генераторы
        - Справочник по TsDocs JSDoc
        - @types
        - JSX
        - Модули ES6
        - Оператор возведения в степень ES7
        - Инструкция for-await-of
        - Новое метасвойство target
        - Динамические выражения импорта
        - “tsc –watch”
        - Оператор утверждения о ненулевом значении
        - Объявления со значениями по умолчанию
        - Опциональная цепочка
        - Оператор объединения с null
        - Типы шаблонных литералов
        - Перегрузка функций
        - Рекурсивные типы
        - Рекурсивные условные типы
        - Поддержка модулей ECMAScript в Node
        - Функции утверждения
        - Вариативные кортежные типы
        - Объектные типы-обёртки

- Ковариантность и контравариантность в TypeScript
  - Необязательные аннотации вариантности для параметров типов
- Индексные сигнатуры с шаблонами строковых литералов
- Оператор satisfies
- Импорт и экспорт только типов
- Объявление using и явное управление ресурсами
  - Объявление await using
- Атрибуты импорта
- Проверка синтаксиса регулярных выражений
- import defer

Введение

Добро пожаловать в «The Concise TypeScript Book»! Это руководство даст вам основные знания и практические навыки для эффективной разработки на TypeScript. Вы познакомитесь с ключевыми понятиями и приёмами написания чистого и надёжного кода. Независимо от того, начинающий вы или опытный разработчик, эта книга послужит и полным руководством, и удобным справочником, который поможет использовать возможности TypeScript в ваших проектах.

В этой книге рассматривается TypeScript 7.0.

Об авторе

Симоне Поджали — опытный Staff-инженер, увлечённый написанием профессионального кода с 1990-х годов. За свою международную карьеру он участвовал во множестве проектов для самых разных клиентов — от стартапов до крупных организаций. Его опыт и преданность делу принесли пользу таким известным компаниям, как HelloFresh, Siemens, O2, Leroy Merlin и Snowplow.

Связаться с Симоне Поджали можно на следующих платформах:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- Электронная почта: gibbok.coding gmail.com

Полный список участников: https://github.com/gibbok/typescript-book/graphs/contributors

Введение в TypeScript

Что такое TypeScript?

TypeScript — это язык программирования со строгой типизацией, основанный на JavaScript. Первоначально он был разработан Андерсом Хейлсбергом в 2012 году, а сейчас развивается и поддерживается Microsoft как проект с открытым исходным кодом.

TypeScript компилируется в JavaScript и может выполняться в любой среде выполнения JavaScript (например, в браузере или в Node.js на сервере).

Он поддерживает несколько парадигм программирования, в том числе функциональное, обобщённое, императивное и объектно-ориентированное программирование, и является компилируемым (транспилируемым) языком, который перед выполнением преобразуется в JavaScript.

Зачем нужен TypeScript?

TypeScript — это язык со строгой типизацией, который помогает предотвращать распространённые ошибки программирования и избегать некоторых видов ошибок времени выполнения ещё до запуска программы.

Язык со строгой типизацией позволяет разработчику задавать различные ограничения и варианты поведения программы в определениях типов данных, упрощая проверку корректности программного обеспечения и предотвращение дефектов. Это особенно ценно для крупных приложений.

Некоторые преимущества TypeScript:

- Статическая типизация, опционально — строгая
- Вывод типов
- Доступ к возможностям ES6 и ES7
- Кроссплатформенная и кроссбраузерная совместимость
- Поддержка инструментов с IntelliSense

TypeScript и JavaScript

Код TypeScript записывается в файлах `.ts` или `.tsx`, а код JavaScript — в файлах `.js` или `.jsx`.

Файлы с расширением `.tsx` или `.jsx` могут содержать расширение синтаксиса JavaScript — JSX, которое используется в React для разработки пользовательских интерфейсов.

С точки зрения синтаксиса TypeScript является типизированным надмножеством JavaScript (ECMAScript 2015). Любой код JavaScript является допустимым кодом TypeScript, но обратное верно не всегда.

Например, рассмотрим функцию в JavaScript-файле с расширением `.js`:

```
const sum = (a, b) => a + b;
```

Эту функцию можно преобразовать и использовать в TypeScript, изменив расширение файла на `.ts`. Однако если ту же функцию снабдить аннотациями типов TypeScript, её нельзя будет выполнить ни в одной среде выполнения JavaScript без компиляции. Следующий код TypeScript приведёт к синтаксической ошибке, если его не скомпилировать:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript был создан для выявления потенциальных ошибок времени выполнения на этапе компиляции: разработчики могут выражать свои намерения с помощью аннотаций типов. Кроме того, благодаря выводу типов TypeScript способен обнаруживать некоторые проблемы даже без явных аннотаций типов. Например, в следующем фрагменте кода не указаны никакие типы TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

В этом случае TypeScript обнаруживает ошибку и сообщает:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Система типов TypeScript во многом учитывает поведение JavaScript во время выполнения. Например, оператор сложения (+), который в JavaScript может выполнять как конкатенацию строк, так и сложение чисел, моделируется в TypeScript таким же образом:

```
const result = '1' + 1; // Результат имеет тип string
```

Команда разработчиков TypeScript приняла осознанное решение отмечать необычное использование JavaScript как ошибку. Например, рассмотрим следующий допустимый код JavaScript:

```
const result = 1 + true; // В JavaScript результат равен 2
```

Однако TypeScript выдаёт ошибку:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Эта ошибка возникает потому, что TypeScript строго контролирует совместимость типов и в данном случае обнаруживает недопустимую операцию между числом и логическим значением.

Генерация кода с помощью TypeScript

У компилятора TypeScript две основные задачи: проверка типов и компиляция в JavaScript. Эти два процесса не

зависят друг от друга. Типы не влияют на выполнение кода в среде выполнения JavaScript, поскольку при компиляции полностью удаляются. TypeScript может сгенерировать JavaScript-код, даже если присутствуют ошибки типов. Ниже приведён пример кода TypeScript с ошибкой типа:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Аргумент типа 'string' нельзя присвоить параметру типа 'number'.
```

Тем не менее он может сгенерировать исполняемый код JavaScript:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Проверить типы TypeScript во время выполнения невозможно. Например:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' обозначает только тип, но здесь используется как значение.
        // ...
    }
};
```

Поскольку после компиляции типы удаляются, выполнить этот код в JavaScript невозможно. Чтобы распознавать типы во время выполнения, необходимо использовать другой механизм. TypeScript предоставляет несколько вариантов, один из распространённых — «дискриминируемое объединение». Например:

```
interface Dog {
    kind: 'dog'; // Размеченное объединение
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Размеченное объединение
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Свойство «kind» представляет собой значение, которое можно использовать во время выполнения, чтобы различать объекты в JavaScript.

Также значение во время выполнения может иметь тип, отличный от указанного в объявлении типа. Например, это может произойти, если разработчик неправильно понял тип API и указал для него неверную аннотацию.

TypeScript является надмножеством JavaScript, поэтому ключевое слово «class» можно использовать как тип и как значение во время выполнения.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};
```

```
const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

В JavaScript у «class» есть свойство «prototype», а оператор «instanceof» позволяет проверить, встречается ли свойство prototype конструктора где-либо в цепочке прототипов объекта.

TypeScript не влияет на производительность во время выполнения, поскольку все типы удаляются. Однако TypeScript создаёт некоторые дополнительные затраты времени на сборку.

Современный JavaScript уже сейчас (понижение версии)

TypeScript может компилировать код в любую выпущенную версию JavaScript, начиная с ECMAScript 3 (1999). Это означает, что TypeScript может транспилировать код, использующий новейшие возможности JavaScript, в более старые версии. Этот процесс называется понижением целевой версии (Downleveling). Благодаря этому можно использовать современный JavaScript, сохраняя максимальную совместимость со старыми средами выполнения.

Важно отметить, что при транспиляции в более старую версию JavaScript TypeScript может сгенерировать код, который приведёт к снижению производительности по сравнению с нативными реализациями.

Ниже перечислены некоторые современные возможности JavaScript, которые можно использовать в TypeScript:

- Модули ECMAScript вместо обратных вызовов «define» в стиле AMD или инструкций «require» CommonJS.
- Классы вместо прототипов.
- Объявление переменных с помощью «let» или «const» вместо «var».
- Цикл «for-of» или метод «.forEach» вместо традиционного цикла «for».
- Стрелочные функции вместо функциональных выражений.
- Деструктурирующее присваивание.
- Сокращённые имена свойств и методов, а также вычисляемые имена свойств.
- Параметры функций по умолчанию.

Используя эти современные возможности JavaScript, разработчики могут писать на TypeScript более выразительный и лаконичный код.

Начало работы с TypeScript

Установка

Visual Studio Code предоставляет отличную поддержку языка TypeScript, но не включает компилятор TypeScript. Чтобы установить компилятор TypeScript, можно воспользоваться менеджером пакетов, например npm или yarn:

```
npm install typescript --save-dev
```

или

```
yarn add typescript --dev
```

Обязательно зафиксируйте созданный lock-файл в репозитории, чтобы все участники команды использовали одну и ту же версию TypeScript.

Для запуска компилятора TypeScript можно использовать следующие команды:

```
npx tsc
```

или

```
yarn tsc
```

Рекомендуется устанавливать TypeScript локально для каждого проекта, а не глобально, поскольку это делает процесс сборки более предсказуемым. Однако для разовых задач можно использовать следующую команду:

```
npx tsc
```

или установить его глобально:

```
npm install -g typescript
```

Если вы используете Microsoft Visual Studio, TypeScript можно получить в виде пакета NuGet для проектов MSBuild. Выполните следующую команду в консоли диспетчера пакетов NuGet:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Во время установки TypeScript устанавливаются два исполняемых файла: «tsc» — компилятор TypeScript и «tsserver» — автономный сервер TypeScript. Автономный сервер содержит компилятор и языковые службы, которые редакторы и IDE могут использовать для интеллектуального автодополнения кода.

Кроме того, доступны несколько транспиляторов, совместимых с TypeScript, например Babel (через плагин) и swc. Эти транспиляторы можно использовать для преобразования кода TypeScript в другие целевые языки или версии.

TypeScript 7.0 был переписан на Go как нативная реализация компилятора и языковой службы. Он использует многопоточность с общей памятью и другие оптимизации, чтобы ускорить полные сборки и функции редактора, сокращая время обратной связи при разработке.

Некоторые возможности повышения производительности TypeScript 7.0 можно настраивать. Проверку типов можно выполнять параллельно в нескольких обработчиках с помощью `--checkers`: большее число обработчиков способно ускорить работу с крупными проектами, но требует больше памяти. Переработанный режим `--watch` улучшает кроссплатформенное отслеживание файлов. В TypeScript 7.0 пока нет API компилятора (по состоянию на июль 2026 года), поэтому инструменты, которым по-прежнему требуется API TypeScript 6.0, могут работать параллельно с TypeScript 7.0 с помощью `@typescript/typescript6` или псевдонимов npm.

Настройка

TypeScript можно настроить с помощью параметров командной строки tsc или специального файла конфигурации tsconfig.json, размещённого в корне проекта.

Чтобы создать файл tsconfig.json с рекомендуемыми настройками, можно использовать следующую команду:

```
tsc --init
```

При локальном выполнении команды `tsc` TypeScript скомпилирует код, используя конфигурацию из ближайшего файла tsconfig.json.

Ниже приведены примеры команд CLI, выполняемых с настройками по умолчанию:

```
tsc main.ts // Компиляция указанного файла (main.ts) в JavaScript
tsc src/*.ts // Компиляция всех файлов .ts в папке 'src' в JavaScript
tsc app.ts util.ts --outfile index.js // Компиляция двух файлов TypeScript (app.ts и util.ts) в один файл JavaScript (index.js)
```

Файл конфигурации TypeScript

Файл tsconfig.json используется для настройки компилятора TypeScript (tsc). Обычно его размещают в корне проекта вместе с файлом `package.json`.

Примечания:

- tsconfig.json допускает комментарии, даже несмотря на формат JSON.

- Рекомендуется использовать этот файл конфигурации вместо параметров командной строки.

По следующей ссылке можно найти полную документацию и её схему:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Ниже приведён список распространённых и полезных настроек:

target

Свойство «target» определяет версию ECMAScript, в которую следует преобразовать или скомпилировать код TypeScript. Для современных браузеров хорошим вариантом является ES6. Примечание: поддержка ES5 была объявлена устаревшей в TypeScript 6.0 и больше не поддерживается в TypeScript 7.0.

lib

Свойство «lib» определяет, какие файлы библиотек следует включать во время компиляции. TypeScript автоматически включает API для возможностей, указанных в свойстве «target», однако при необходимости можно исключить или выбрать конкретные библиотеки. Например, при работе над серверным проектом можно исключить библиотеку «DOM», которая полезна только в среде браузера.

strict

Параметр «strict» повышает безопасность типов за счёт включения более строгих проверок. Начиная с TypeScript 6.0 он включён по умолчанию; в противном случае в файле tsconfig.json следует явно задать для него значение true. Включение «strict» позволяет TypeScript выполнять следующие действия:

- Генерировать код с директивой «use strict» для каждого исходного файла.
- Учитывать «null» и «undefined» при проверке типов.
- Запрещать использование типа «any», если аннотации типов отсутствуют.
- Сообщать об ошибке при использовании выражения «this», для которого в противном случае подразумевался бы тип «any».

module

Свойство «module» задаёт систему модулей, поддерживаемую скомпилированной программой. Во время выполнения загрузчик модулей находит и выполняет зависимости в соответствии с указанной системой модулей.

Наиболее распространённые загрузчики модулей в JavaScript — Node.js CommonJS для серверных приложений и RequireJS для модулей AMD в браузерных веб-приложениях. TypeScript может генерировать код для различных систем модулей, включая UMD, System, ESNext, ES2015/ES6 и ES2020. Систему модулей следует выбирать с учётом целевой среды и доступного в ней механизма загрузки модулей.

Примечание: поддержка старых систем модулей (AMD, UMD, SystemJS) была объявлена устаревшей в TypeScript 6.0 и больше не предоставляется в TypeScript 7.0.

moduleResolution

Свойство «moduleResolution» определяет стратегию разрешения модулей. Для современного кода TypeScript используйте «nodenext» или «bundler». Стратегия «classic» используется только в старых версиях TypeScript (до 1.6).

esModuleInterop

Свойство «esModuleInterop» разрешает импорт по умолчанию из модулей CommonJS, в которых экспорт не выполнялся через свойство «default»; это свойство предоставляет прослойку, обеспечивающую совместимость в сгенерированном JavaScript. После включения этого параметра можно использовать `import MyLibrary from "my-library"` вместо `import * as MyLibrary from "my-library"`.

Изначально «esModuleInterop» был необязательным параметром во избежание обратно несовместимых изменений, но уже давно является рекомендуемой настройкой по умолчанию. Его отключение может привести к неочевидным проблемам во время выполнения при совместном использовании CommonJS и ESM. Примечание: начиная с TypeScript 6.0 это более безопасное поведение совместимости включено всегда.

В TypeScript 6.0 некоторые старые параметры конфигурации и формы синтаксиса были объявлены

устаревшими либо переведены в категорию устаревшего поведения. В TypeScript 7.0 они становятся ошибками или не оказывают никакого действия.

Устаревшие возможности, использование которых теперь приводит к критическим ошибкам и не оказывает никакого воздействия:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- отключение `esModuleInterop` или `allowSyntheticDefaultImports`
- отключение `alwaysStrict`
- ключевое слово `module` в объявлениях пространств имён
- `asserts` в импортах
- `/// <reference no-default-lib />` при использовании `skipDefaultLibCheck`
- пути к файлам в CLI при наличии локального `tsconfig.json`, если не используется `--ignoreConfig`

jsx

Свойство “jsx” применяется только к файлам .tsx, используемым в ReactJS, и управляет тем, как конструкции JSX компилируются в JavaScript. Часто используется вариант “preserve”, при котором код компилируется в файл .jsx с сохранением JSX без изменений, чтобы его можно

было передать другим инструментам, таким как Babel, для дальнейших преобразований.

skipLibCheck

Свойство “skipLibCheck” запрещает TypeScript выполнять проверку типов во всех импортированных сторонних пакетах. Это свойство сокращает время компиляции проекта. TypeScript по-прежнему будет проверять ваш код на соответствие определениям типов, предоставленным этими пакетами.

files

Свойство “files” указывает компилятору список файлов, которые всегда должны быть включены в программу.

include

Свойство “include” указывает компилятору список файлов, которые требуется включить. Это свойство допускает шаблоны, подобные glob: например, *“*” для любого подкаталога, ””* для любого имени файла и “?” для необязательных символов.

exclude

Свойство “exclude” указывает компилятору список файлов, которые не следует включать в компиляцию. Сюда могут входить такие файлы, как “node_modules”, или тестовые файлы. Примечание: tsconfig.json допускает комментарии.

importHelpers

TypeScript использует вспомогательный код при генерации кода для некоторых расширенных возможностей JavaScript или возможностей, преобразуемых для более ранних версий. По умолчанию эти вспомогательные функции дублируются в использующих их файлах. Вместо этого параметр `importHelpers` импортирует их из модуля `tslib`, делая выходной JavaScript-код более эффективным.

Рекомендации по переходу на TypeScript

Для крупных проектов рекомендуется постепенный переход, при котором код на TypeScript и JavaScript поначалу будет сосуществовать. Только небольшие проекты можно перевести на TypeScript за один раз.

Первый шаг этого перехода — внедрить TypeScript в процесс сборки. Это можно сделать с помощью параметра компилятора “allowJs”, который позволяет файлам .ts и .tsx сосуществовать с имеющимися файлами JavaScript. Поскольку TypeScript использует тип “any” для переменной, когда не может вывести тип из файлов JavaScript, в начале миграции рекомендуется отключить “noImplicitAny” в параметрах компилятора.

Второй шаг — убедиться, что тесты JavaScript работают вместе с файлами TypeScript, чтобы можно было запускать тесты по мере преобразования каждого модуля. Если вы используете Jest, рассмотрите возможность применения `ts-jest`, который позволяет тестировать проекты TypeScript с помощью Jest.

Третий шаг — добавить в проект объявления типов для сторонних библиотек. Эти объявления могут поставляться вместе с библиотеками или находиться в DefinitelyTyped. Их можно найти с помощью https://www.typescriptlang.org/dt/search и установить следующим образом:

```
npm install --save-dev @types/package-name
```

или

```
yarn add --dev @types/package-name
```

Четвёртый шаг — выполнять миграцию модуль за модулем, следуя восходящему подходу и начиная с листьев графа зависимостей. Идея состоит в том, чтобы начать преобразование с модулей, которые не зависят от других модулей. Для визуализации графов зависимостей можно использовать инструмент “madge”.

Хорошими кандидатами для первоначального преобразования являются служебные функции и код, связанный с внешними API или спецификациями. Можно автоматически создавать определения типов TypeScript из контрактов Swagger, схем GraphQL или JSON для включения в проект.

Если спецификации или официальные схемы недоступны, типы можно создать на основе необработанных данных, например JSON, возвращаемого сервером. Однако рекомендуется создавать типы на основе спецификаций, а не данных, чтобы не упустить пограничные случаи.

Во время миграции воздержитесь от рефакторинга кода и сосредоточьтесь исключительно на добавлении типов в модули.

Пятый шаг — включить “noImplicitAny”, что потребует, чтобы все типы были известны и определены, и улучшит работу с TypeScript в проекте.

Во время миграции можно использовать директиву `@ts-check`, которая включает проверку типов TypeScript в файле JavaScript. Эта директива предоставляет менее строгую версию проверки типов и на начальном этапе может использоваться для выявления проблем в файлах JavaScript. Если файл содержит `@ts-check`, TypeScript попытается вывести определения с помощью комментариев в стиле JSDoc. Однако следует использовать аннотации JSDoc только на самом раннем этапе миграции.

Рассмотрите возможность сохранить для `noEmitOnError` в файле tsconfig.json значение по умолчанию false. Это позволит создавать исходный код JavaScript, даже если были обнаружены ошибки.

Изучение системы типов

Языковая служба TypeScript

Языковая служба TypeScript, также известная как tsserver, предоставляет различные возможности, включая сообщения об ошибках, диагностику, компиляцию при сохранении, переименование, переход к определению, списки автодополнения, подсказки по сигнатурам и

многое другое. Она преимущественно используется интегрированными средами разработки (IDE) для поддержки IntelliSense. Служба беспрепятственно интегрируется с Visual Studio Code и используется такими инструментами, как Conquer of Completion (Coc).

Разработчики могут использовать специальный API и создавать собственные плагины языковой службы, чтобы улучшить процесс редактирования TypeScript. Это может быть особенно полезно для реализации специальных возможностей линтинга или включения автодополнения для пользовательского языка шаблонов.

Примером реального специализированного плагина является “typescript-styled-plugin”, который сообщает о синтаксических ошибках и поддерживает IntelliSense для свойств CSS в стилизованных компонентах.

Дополнительную информацию и руководства по быстрому началу работы можно найти в официальной вики TypeScript на GitHub: https://github.com/microsoft/TypeScript/wiki/

Структурная типизация

TypeScript основан на структурной системе типов. Это означает, что совместимость и эквивалентность типов определяются фактической структурой или определением типа, а не его именем или местом объявления, как в номинативных системах типов, таких как C# или C.

Структурная система типов TypeScript разработана с учётом того, как динамическая утиная типизация

JavaScript работает во время выполнения.

Следующий пример является допустимым кодом TypeScript. Как можно заметить, “X” и “Y” имеют один и тот же член “a”, хотя им присвоены разные имена при объявлении. Типы определяются их структурами, а поскольку в данном случае структуры одинаковы, типы совместимы и допустимы.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Допустимо
```

Основные правила сравнения в TypeScript

Процесс сравнения в TypeScript является рекурсивным и выполняется для типов, вложенных на любом уровне.

Тип “X” совместим с “Y”, если “Y” имеет как минимум те же члены, что и “X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Допустимо, поскольку содержит как минимум те же члены, что и X
const r: X = y;
```

Параметры функций сравниваются по типам, а не по именам:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Допустимо
x = y; // Допустимо
```

Возвращаемые типы функций должны совпадать:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Недопустимо
x = y; // Недопустимо
```

Возвращаемый тип исходной функции должен быть подтипом возвращаемого типа целевой функции:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Допустимо
y = x; // Недопустимо: отсутствует член b
```

Отбрасывать параметры функции разрешено, поскольку это распространённая практика в JavaScript, например при использовании “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Таким образом, следующие объявления типов полностью допустимы:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Отсутствует параметр b
y = x; // Допустимо
```

Любые дополнительные необязательные параметры исходного типа допустимы:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Допустимо
x = y; //Допустимо
```

Любые необязательные параметры целевого типа, которым не соответствуют параметры исходного типа, допустимы и не приводят к ошибке:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Допустимо
x = y; // Допустимо
```

Остаточный параметр рассматривается как бесконечная последовательность необязательных параметров:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //допустимо
```

Функции с перегрузками допустимы, если сигнатура перегрузки совместима с сигнатурой реализации:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Допустимо
x('a', 1); // Допустимо
```

```
function y(a: string): void; // Недопустимо: несовместимо с сигнатурой реализации
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Сравнение параметров функций считается успешным, если исходные и целевые параметры могут быть присвоены супертипам или подтипам (бивариантность).

```
// Надтип
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Подтип
class Y extends X {}
// Подтип
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Бивариантность допускает надтипы
console.log(getA(new X('x'))); // Допустимо
console.log(getA(new Y('Y'))); // Допустимо
console.log(getA(new Z('z'))); // Допустимо
```

Перечисления совместимы с числами и наоборот, однако сравнение значений из перечислений разных типов недопустимо.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Допустимо
const ya: Y = 0; // Допустимо
X.A === Y.A; // Недопустимо
```

При проверке совместимости экземпляров классов учитываются их закрытые и защищённые члены:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Недопустимо
```

При сравнении не учитываются различия в иерархии наследования, например:

```
class X {
    public a: string;
    constructor(value: string) {
```

```
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Допустимо
x === z; // Допустимо, даже если z принадлежит другой иерархии наследования
```

Обобщённые типы сравниваются по их структурам на основе результирующего типа после применения параметра типа. Сравнивается только конечный результат как необобщённый тип.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Недопустимо, поскольку аргумент типа используется в итоговой структуре

interface X<T> {}
const x: X<number> = 1;
```

```
const y: X<string> = 'a';
x === y; // Допустимо, поскольку аргумент типа не используется в итоговой структуре
```

Если аргументы обобщённых типов не указаны, все неуказанные аргументы рассматриваются как типы “any”:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Допустимо
```

Помните:

```
let a: number = 1;
let b: number = 2;
a = b; // Допустимо: любой тип присваиваем самому себе

let c: any;
c = 1; // Допустимо: все типы присваиваемы типу any

let d: unknown;
d = 1; // Допустимо: все типы присваиваемы типу unknown

let e: unknown;
let e1: unknown = e; // Допустимо: unknown присваиваем только самому себе и any
let e2: any = e; // Допустимо
let e3: number = e; // Недопустимо

let f: never;
f = 1; // Недопустимо: ничто не присваиваемо типу never

let g: void;
let g1: any;
```

```
g = 1; // Недопустимо: void нельзя присвоить ничему и ему нельзя присвоить ничего, кроме any
g = g1; // Допустимо
```

Обратите внимание, что при включённом параметре “strictNullChecks” типы “null” и “undefined” обрабатываются аналогично “void”; в противном случае — аналогично “never”.

Типы как множества

В TypeScript тип представляет собой множество возможных значений. Это множество также называется областью определения типа. Каждое значение типа можно рассматривать как элемент множества. Тип задаёт ограничения, которым должен удовлетворять каждый элемент множества, чтобы считаться его членом. Основная задача TypeScript — проверять, является ли одно множество подмножеством другого.

TypeScript поддерживает различные виды множеств:

Термин теории множеств	**TypeScript**	**Примечания**
Пустое множество	never	“never” не содержит ничего, кроме самого себя
Одноэлементное множество	undefined / null / literal type	
Конечное множество	boolean / union	
Бесконечное множество	string / number / object	

Универсальное множество	any / unknown	Каждый элемент принадлежит “any”, и каждое множество является его подмножеством / “unknown” — типобезопасный аналог “any”

Ниже приведено несколько примеров:

TypeScript	**Термин теории множеств**	**Пример**
never	∅ (пустое множество)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Литеральный тип	Одноэлементное множество	type X = ‘X’; type Y = 7;
Значение, присваиваемое T	Значение ∈ T (принадлежит)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T1 присваивается T2	T1 ⊆ T2 (подмножество)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (подмножество)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (объединение)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (пересечение)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }

unknown	Универсальное множество	const x: unknown = 1

Объединение (T1 | T2) создаёт более широкое множество (оба множества):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Допустимо
```

Пересечение (T1 & T2) создаёт более узкое множество (только общие элементы):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Недопустимо
const j: XY = { a: 'a', b: 'b' }; // Допустимо
```

В этом контексте ключевое слово `extends` можно рассматривать как «является подмножеством». Оно задаёт ограничение для типа. Когда `extends` используется с обобщённым типом, оно ограничивает параметр обобщённого типа более конкретным типом.

Обратите внимание, что здесь `extends` не имеет отношения к наследованию классов в смысле ООП.

TypeScript работает со структурными типами и не имеет строгой номинальной иерархии. Как показано в примере ниже, два типа могут пересекаться, хотя ни один из них не является подтипом другого, поскольку TypeScript учитывает структуру, или форму, объектов.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Допустимо
```

Задание типа: объявления типов и утверждения типов

В TypeScript тип можно задать разными способами:

Объявление типа

В следующем примере мы используем x: X (“: Type”), чтобы объявить тип переменной x.

```
type X = {
    a: string;
};

// Объявление типа
const x: X = {
    a: 'a',
};
```

Если переменная не соответствует указанному формату, TypeScript сообщит об ошибке. Например:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
    b: 'b', // Ошибка: объектный литерал может содержать только известные свойства
};
```

Утверждение типа

Утверждение можно добавить с помощью ключевого слова `as`. Оно сообщает компилятору, что разработчик располагает дополнительной информацией о типе, и подавляет возможные ошибки.

Например:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

В приведённом выше примере с помощью ключевого слова as утверждается, что объект x имеет тип X. Это сообщает компилятору TypeScript, что объект соответствует указанному типу, даже если у него есть дополнительное свойство b, отсутствующее в определении типа.

Утверждения типов полезны в ситуациях, когда необходимо указать более конкретный тип, особенно при работе с DOM. Например:

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

Здесь утверждение типа as HTMLInputElement сообщает TypeScript, что результат getElementById следует рассматривать как HTMLInputElement. Утверждения типов также можно использовать для переназначения ключей, как показано в приведённом ниже примере с шаблонными литералами:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
```

```
};
type Y = J<X>;
```

В этом примере тип `J<Type>` использует сопоставляемый тип с шаблонным литералом для переназначения ключей Type. Он создаёт новые свойства, добавляя к каждому ключу “prefix_”, а их соответствующими значениями становятся функции, возвращающие исходные значения свойств.

Стоит отметить, что при использовании утверждения типа TypeScript не выполняет проверку избыточных свойств. Поэтому, если структура объекта известна заранее, обычно предпочтительнее использовать объявление типа.

Внешние объявления

Внешние объявления — это файлы, описывающие типы для кода JavaScript; их имена имеют формат `.d.ts`. Обычно они импортируются и используются для аннотирования существующих библиотек JavaScript или добавления типов к существующим файлам JS в проекте.

Типы для многих распространённых библиотек можно найти здесь: https://github.com/DefinitelyTyped/DefinitelyTyped/

и установить следующим образом:

```
npm install --save-dev @types/library-name
```

Собственные внешние объявления можно импортировать с помощью ссылки с «тремя косыми чертами»:

```
/// <reference path="./library-types.d.ts" />
```

Внешние объявления можно использовать даже в файлах JavaScript с помощью `// @ts-check`.

Ключевое слово `declare` позволяет задавать определения типов для существующего кода JavaScript без его импорта, выступая в роли заполнителя для типов из другого файла или глобальной области.

Проверка свойств и проверка избыточных свойств

TypeScript основан на структурной системе типов, однако проверка избыточных свойств — это возможность TypeScript, которая позволяет проверить, содержит ли объект в точности те свойства, которые указаны в типе.

Проверка избыточных свойств выполняется, например, при присваивании объектных литералов переменным или при их передаче в качестве аргументов функции.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Допустимо благодаря структурной типизации
const w: X = { a: 'a', b: 'b' }; // Недопустимо из-за проверки
лишних свойств
```

Слабые типы

Тип считается слабым, если он содержит только набор необязательных свойств:

```ts
type X = {
    a?: string;
    b?: string;
};
```

TypeScript считает ошибкой присваивание слабому типу значения, с которым у него нет общих свойств. Например, следующий код приводит к ошибке:

```ts
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Недопустимо
```

Хотя это не рекомендуется, при необходимости данную проверку можно обойти с помощью утверждения типа:

```ts
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Допустимо
```

Либо добавив unknown в индексную сигнатуру слабого типа:

```ts
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Допустимо
```

Строгая проверка объектных литералов (свежесть)

Строгая проверка объектных литералов, иногда называемая «свежестью», — это возможность TypeScript, которая помогает обнаруживать избыточные свойства или свойства с опечатками, которые в противном случае остались бы незамеченными при обычных проверках структурной типизации.

При создании объектного литерала компилятор TypeScript считает его «свежим». Если объектный литерал присваивается переменной или передаётся как параметр, TypeScript выдаёт ошибку, если объектный литерал содержит свойства, отсутствующие в целевом типе.

Однако «свежесть» исчезает, когда объектный литерал расширяется или используется утверждение типа.

Рассмотрим несколько примеров:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Проверка свежести: недопустимое присваивание
var y: Y;
y = { a: 'a', bx: 'bx' }; // Проверка свежести: недопустимое присваивание

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Расширение: ошибок нет, типы структурно совместимы
```

```
fn({ a: 'a', bx: 'b' }); // Проверка свежести: недопустимый аргумент

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Расширение: проверка свежести не выполняется
```

Вывод типов

TypeScript может выводить типы, если аннотация не указана при:

- Инициализации переменной.
- Инициализации члена.
- Установке значений параметров по умолчанию.
- Определении возвращаемого типа функции.

Например:

```
let x = 'x'; // Выведен тип string
```

Компилятор TypeScript анализирует значение или выражение и определяет его тип на основе доступной информации.

Более сложные случаи вывода типов

Если при выводе типов используется несколько выражений, TypeScript ищет «наилучшие общие типы». Например:

```
let x = [1, 'x', 1, null]; // Выведен тип: (string | number | null)[]
```

Если компилятор не может найти наилучшие общие типы, он возвращает тип-объединение. Например:

```
let x = [new RegExp('x'), new Date()]; // Выведен тип: (RegExp | Date)[]
```

TypeScript использует «контекстную типизацию» на основе расположения переменной для вывода типов. В следующем примере компилятор знает, что `e` имеет тип `MouseEvent`, благодаря типу события `click`, определённому в файле lib.d.ts, который содержит внешние объявления для различных распространённых конструкций JavaScript и DOM:

```
window.addEventListener('click', function (e) {}); // Для e выведен тип MouseEvent
```

Расширение типов

Расширение типов — это процесс, при котором TypeScript назначает тип переменной, инициализированной без аннотации типа. Этот процесс допускает переход от узких типов к более широким, но не наоборот. В следующем примере:

```
let x = 'x'; // TypeScript выводит широкий тип string
let y: 'y' | 'x' = 'y'; // Тип y — объединение литеральных типов
y = x; // Недопустимо: тип 'string' нельзя присвоить типу '"x" | "y"'.
```

На основе единственного значения, указанного при инициализации (`x`), TypeScript назначает переменной `x` тип `string`; это пример расширения.

TypeScript позволяет управлять процессом расширения, например с помощью “const”.

Const

Использование ключевого слова `const` при объявлении переменной приводит к выводу более узкого типа в TypeScript.

Например:

```
const x = 'x'; // TypeScript выводит для x более узкий тип 'x'
let y: 'y' | 'x' = 'y';
y = x; // Допустимо: для x выведен тип 'x'
```

При использовании `const` для объявления переменной x её тип сужается до конкретного литерального значения ‘x’. Поскольку тип x сужен, его можно присвоить переменной y без ошибок. Тип можно вывести, поскольку переменным `const` нельзя повторно присваивать значения, поэтому их тип можно сузить до конкретного литерального типа — в данном случае до литерального типа ‘x’.

Модификатор const для параметров типов

Начиная с TypeScript 5.0, для параметра обобщённого типа можно указать атрибут `const`. Это позволяет вывести наиболее точный возможный тип. Рассмотрим пример без использования `const`:

```
function identity<T>(value: T) {
    // Здесь нет const
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Выведен тип: {
a: string; b: string; }
```

Как видно, для свойств `a` и `b` выводится тип `string`.

Теперь рассмотрим различие с версией, использующей `const`:

```
function identity<const T>(value: T) {
    // Использование модификатора const для параметров типа
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Выведен тип: {
a: "a"; b: "b"; }
```

Теперь видно, что для свойств `a` и `b` выводятся строковые литералы, а не просто типы `string`.

Утверждение const

Эта возможность позволяет объявить переменную с более точным литеральным типом на основе её начального значения, сообщая компилятору, что это значение следует рассматривать как неизменяемый литерал. Ниже приведено несколько примеров:

Для отдельного свойства:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Для всего объекта:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Это может быть особенно полезно при определении типа кортежа:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Кортеж readonly [1, 2, 3]
```

Явная аннотация типа

Можно явно указать тип. В следующем примере свойство `x` имеет тип `number`:

```
const v = {
    x: 1, // Выведен тип: number (расширение)
};
v.x = 3; // Допустимо
```

Аннотацию типа можно сделать более конкретной, используя объединение литеральных типов:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // Теперь x имеет объединение литеральных типов: 1 |
2 | 3
};
v.x = 3; // Допустимо
v.x = 100; // Недопустимо
```

Сужение типов

Сужение типов — это процесс в TypeScript, при котором общий тип сужается до более конкретного. Это происходит, когда TypeScript анализирует код и определяет, что некоторые условия или операции позволяют уточнить информацию о типе.

Сужение типов может происходить разными способами, в том числе следующими:

Условия

С помощью условных конструкций, таких как `if` или `switch`, TypeScript может сузить тип на основе результата условия. Например:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // Тип number, суженный условием
}
```

Выбрасывание исключения или возврат

Выбрасывание исключения или досрочный возврат из ветви можно использовать, чтобы помочь TypeScript сузить тип. Например:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

К другим способам сужения типов в TypeScript относятся:

- Оператор `instanceof`: используется для проверки того, является ли объект экземпляром определённого класса.
- Оператор `in`: используется для проверки наличия свойства в объекте.
- Оператор `typeof`: используется для проверки типа значения во время выполнения.

- Встроенные функции, такие как `Array.isArray()`: используются для проверки того, является ли значение массивом.

Дискриминируемое объединение

«Дискриминируемое объединение» — это шаблон в TypeScript, в котором к объектам добавляется явная «метка», позволяющая различать типы внутри объединения. Этот шаблон также называют «помеченным объединением». В следующем примере «метка» представлена свойством «type»:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // тип A
        case 'type_b':
            return input.value + 'extra'; // тип B
    }
};
```

Пользовательские защитники типа

В случаях, когда TypeScript не может определить тип, можно написать вспомогательную функцию, известную как «пользовательский защитник типа». В следующем примере мы воспользуемся предикатом типа, чтобы сузить тип после применения фильтрации:

```ts
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Тип (string | null)[]: TypeScript не смог правильно вывести тип

const isValid = (item: string | null): item is string => item !== null; // Пользовательская защита типа

const r2 = data.filter(isValid); // Теперь тип корректен: string[]; предикат защиты типа позволил сузить тип
```

Сужение с помощью switch (true)

В TypeScript 5.3 появилось сужение с помощью switch (true), позволяющее заменить громоздкие цепочки if/else конструкцией switch (true) с логическими условиями. Оно улучшает читаемость и при этом сохраняет сужение типов. Это похоже на сопоставление с образцом, но проще.

```ts
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Примитивные типы

TypeScript поддерживает 7 примитивных типов. Примитивный тип данных — это тип, который не является

объектом и не имеет связанных с ним методов. В TypeScript все примитивные типы неизменяемы, то есть после присваивания их значения нельзя изменить.

string

Примитивный тип `string` хранит текстовые данные, а его значение всегда заключено в двойные или одинарные кавычки.

```
const x: string = 'x';
const y: string = 'y';
```

Строки могут занимать несколько строк, если окружить их символами обратной кавычки (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Тип данных `boolean` в TypeScript хранит двоичное значение: `true` или `false`.

```
const isReady: boolean = true;
```

number

Тип данных `number` в TypeScript представлен 64-битным числом с плавающей точкой. Тип `number` может представлять целые и дробные числа. TypeScript также поддерживает шестнадцатеричные, двоичные и восьмеричные числа, например:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Шестнадцатеричное число начинается с 0x
const binary: number = 0b1010; // Двоичное число начинается с 0b
const octal: number = 0o633; // Восьмеричное число начинается с 0o
```

bigint

Тип `bigint` представляет целые значения, которые могут превышать максимальное безопасное целое число, поддерживаемое типом `number`, — 2^53 - 1.

Значение `bigint` можно создать, вызвав встроенную функцию `BigInt()` или добавив `n` в конец любого целочисленного литерала:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Примечания:

- Значения `bigint` нельзя смешивать с `number` и нельзя использовать со встроенным объектом `Math`; их необходимо привести к одному типу.
- Значения `bigint` доступны, только если в конфигурации указана целевая версия ES2020 или выше.

Symbol

Символы — это уникальные идентификаторы, которые можно использовать в качестве ключей свойств объектов во избежание конфликтов имён.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null и undefined

Типы `null` и `undefined` означают отсутствие значения.

Тип `undefined` означает, что значение не присвоено или не инициализировано, либо указывает на непреднамеренное отсутствие значения.

Тип `null` означает, что нам известно, что поле не имеет значения, поэтому оно недоступно; этот тип указывает на преднамеренное отсутствие значения.

Массив

`array` — это тип данных, который может хранить несколько значений одного или разных типов. Его можно определить с помощью следующего синтаксиса:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Объединение
```

TypeScript поддерживает массивы только для чтения со следующим синтаксисом:

```
const x: readonly string[] = ['a', 'b']; // Модификатор readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Недопустимо
```

TypeScript поддерживает кортежи и кортежи только для чтения:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Тип данных any представляет буквально «любое» значение и используется по умолчанию, когда TypeScript не может вывести тип или тип не указан.

При использовании any компилятор TypeScript пропускает проверку типов, поэтому при использовании any безопасность типов отсутствует. Как правило, не используйте any, чтобы заглушить ошибку компилятора; вместо этого сосредоточьтесь на исправлении ошибки, поскольку использование any позволяет нарушать контракты и лишает преимуществ автодополнения TypeScript.

Тип any может быть полезен при постепенной миграции с JavaScript на TypeScript, поскольку позволяет заглушать ошибки компилятора.

В новых проектах используйте параметр конфигурации TypeScript `noImplicitAny`, который заставляет TypeScript сообщать об ошибках там, где используется или выводится `any`.

Тип `any` обычно становится источником ошибок, способных скрыть реальные проблемы с типами. По возможности избегайте его использования.

Аннотации типов

К переменным, объявленным с помощью `var`, `let` и `const`, можно при необходимости добавить тип:

```
const x: number = 1;
```

TypeScript хорошо справляется с выводом типов, особенно простых, поэтому в большинстве случаев такие объявления не нужны.

В функциях к параметрам можно добавлять аннотации типов:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Ниже приведён пример с анонимной функцией (также называемой лямбда-функцией):

```
const sum = (a: number, b: number) => a + b;
```

Эти аннотации можно не указывать, если у параметра есть значение по умолчанию:

```
const sum = (a = 10, b: number) => a + b;
```

К функциям можно добавлять аннотации возвращаемого типа:

```
const sum = (a = 10, b: number): number => a + b;
```

Это особенно полезно для более сложных функций, поскольку указание возвращаемого типа до написания реализации помогает продумать функцию.

В целом стоит добавлять аннотации типов к сигнатурам, но не к локальным переменным в теле функции, и всегда указывать типы для объектных литералов.

Необязательные свойства

В объекте свойство можно сделать необязательным, добавив вопросительный знак ? в конец его имени:

```
type X = {
    a: number;
    b?: number; // Необязательное свойство
};
```

Для необязательного свойства можно указать значение по умолчанию:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Свойства только для чтения

Запретить запись в свойство можно с помощью модификатора `readonly`, который гарантирует невозможность перезаписи свойства, но не обеспечивает полной неизменяемости:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Индексные сигнатуры

В TypeScript в индексных сигнатурах можно использовать `string`, `number` и `symbol`:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Тот же результат, что и k[1]
```

Обратите внимание, что JavaScript автоматически преобразует индекс типа `number` в индекс типа `string`,

поэтому `k[1]` и `k["1"]` возвращают одно и то же значение.

Расширение типов

Можно расширить `interface` (скопировать члены из другого типа):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Также можно расширить сразу несколько типов:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Ключевое слово `extends` работает только с интерфейсами и классами; для типов используйте пересечение:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Тип можно расширить с помощью интерфейса, но не наоборот:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Литеральные типы

Литеральный тип — это множество из одного элемента внутри составного типа; он задаёт строго определённое значение, являющееся примитивом JavaScript.

Литеральные типы в TypeScript бывают числовыми, строковыми и логическими.

Примеры литералов:

```
const a = 'a'; // Строковый литеральный тип
const b = 1; // Числовой литеральный тип
const c = true; // Логический литеральный тип
```

Строковые, числовые и логические литеральные типы используются в объединениях, защитниках типа и псевдонимах типов. В следующем примере показан псевдоним типа-объединения. `O` состоит только из указанных значений; никакая другая строка недопустима:

```
type O = 'a' | 'b' | 'c';
```

Вывод литеральных типов

Вывод литеральных типов — это возможность TypeScript выводить тип переменной или параметра на основе его значения.

В следующем примере видно, что TypeScript считает тип `x` литеральным, поскольку его значение нельзя изменить позднее, тогда как для `y` выводится тип string, поскольку его значение позднее можно изменить.

```
const x = 'x'; // Литеральный тип 'x', поскольку это значение нельзя изменить
let y = 'y'; // Тип string, поскольку это значение можно изменить
```

В следующем примере видно, что для `o.x` был выведен тип `string` (а не литеральный тип `a`), поскольку TypeScript считает, что значение можно изменить позднее.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Это более широкий тип string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x));  // Аргумент типа 'string' нельзя присвоить параметру типа 'X'
```

Как видно, при передаче `o.x` в `fn` возникает ошибка, поскольку X — более узкий тип.

Эту проблему можно решить с помощью утверждения типа `const` или `X`:

```
let o = {
    x: 'a' as const,
};
```

или:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` — это параметр компилятора TypeScript, включающий строгую проверку null. Когда этот параметр включён, переменным и параметрам можно присваивать `null` или `undefined`, только если они были явно объявлены как имеющие соответствующий тип с помощью типа-объединения `null | undefined`. Если переменная или параметр не объявлены явно как допускающие null, TypeScript выдаст ошибку, чтобы предотвратить возможные ошибки во время выполнения.

Перечисления

В TypeScript `enum` — это набор именованных константных значений.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Перечисления можно определять разными способами:

Числовые перечисления

В TypeScript числовое перечисление — это перечисление, в котором каждой константе присвоено числовое значение, по умолчанию начиная с 0.

```
enum Size {
    Small, // значение начинается с 0
    Medium,
    Large,
}
```

Можно задать пользовательские значения, присвоив их явно:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Строковые перечисления

В TypeScript строковое перечисление — это перечисление, в котором каждой константе присвоено строковое значение.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Примечание: TypeScript позволяет использовать разнородные перечисления, в которых могут одновременно присутствовать строковые и числовые члены.

Константные перечисления

Константное перечисление в TypeScript — это особый вид перечисления, все значения которого известны во время компиляции и подставляются непосредственно в места использования, что делает код эффективнее.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Будет скомпилировано в:

```
console.log('EN' /* Language.English */);
```

Примечания: Значения константных перечислений жёстко заданы, а сами перечисления стираются из скомпилированного кода, что может быть эффективнее в автономных библиотеках, но в целом нежелательно. Кроме того, константные перечисления не могут содержать вычисляемые члены.

Обратное сопоставление

В TypeScript обратное сопоставление в перечислениях означает возможность получить имя члена перечисления по его значению. По умолчанию члены перечисления имеют прямое сопоставление имени со значением, однако обратное сопоставление можно создать, явно задав значения для каждого члена. Обратные сопоставления полезны, когда требуется найти член перечисления по его значению или перебрать все члены перечисления.

Обратите внимание: обратные сопоставления создаются только для числовых членов перечисления, а для строковых членов они не создаются вовсе.

Следующее перечисление:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Компилируется в:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Таким образом, сопоставление значений с ключами работает для числовых членов перечисления, но не для строковых:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A
```

```
const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  // Элемент неявно имеет тип
'any', поскольку индексное выражение не имеет тип 'number'.
```

Внешние перечисления

Внешнее перечисление в TypeScript — это вид перечисления, определённый в файле деклараций (*.d.ts) без связанной с ним реализации. Оно позволяет определить набор именованных констант, которые можно безопасно с точки зрения типов использовать в разных файлах без необходимости импортировать детали реализации в каждый файл.

Вычисляемые и константные члены

В TypeScript вычисляемый член — это член перечисления, значение которого вычисляется во время выполнения, тогда как константный член — это член, значение которого задаётся во время компиляции и не может быть изменено во время выполнения. Вычисляемые члены разрешены в обычных перечислениях, а константные — как в обычных, так и в константных перечислениях.

```
// Константные члены
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6, вычисляется во время компиляции
```

```
// Вычисляемые члены
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // случайное число, сгенерированное во время выполнения
```

Перечисления представлены объединениями, состоящими из типов их членов. Значения каждого члена могут определяться константными или неконстантными выражениями, при этом членам с константными значениями назначаются литеральные типы. Для примера рассмотрим объявление типа E и его подтипов E.A, E.B и E.C. В данном случае E представляет объединение E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Числовой литерал
    B = 'bar', // Строковый литерал
    C = identity(42), // Непрозрачное вычисляемое значение
}

console.log(E.C); //42
```

Сужение типов

Сужение типов в TypeScript — это процесс уточнения типа переменной внутри условного блока. Это полезно при работе с типами-объединениями, где переменная может иметь более одного типа.

TypeScript распознаёт несколько способов сужения типа:

Защитники типа typeof

Защитник типа typeof — это особый защитник типа в TypeScript, который проверяет тип переменной на основе её встроенного типа JavaScript.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x имеет тип number
    }
    return -1;
};
```

Сужение по истинности

Сужение по истинности в TypeScript проверяет, является ли значение переменной истинным или ложным в логическом контексте, и соответствующим образом сужает её тип.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Сужение по равенству

Сужение по равенству в TypeScript проверяет, равна ли переменная определённому значению, и соответствующим образом сужает её тип.

Оно используется совместно с инструкциями `switch` и операторами равенства, такими как `===`, `!==`, `==` и `!=`, для сужения типов.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Сужение с помощью оператора in

Сужение с помощью оператора `in` в TypeScript позволяет сузить тип переменной на основе наличия свойства в её типе.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Сужение с помощью instanceof

Сужение с помощью оператора `instanceof` в TypeScript позволяет сузить тип переменной на основе функции-конструктора, проверяя, является ли объект экземпляром определённого класса или интерфейса.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Присваивания

Сужение типов в TypeScript с помощью присваиваний позволяет сузить тип переменной на основе присвоенного ей значения. Когда переменной присваивается значение, TypeScript выводит её тип на основе этого значения и сужает тип переменной в соответствии с выведенным типом.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Анализ потока управления

Анализ потока управления в TypeScript — это способ статического анализа потока кода для вывода типов переменных, позволяющий компилятору при необходимости сужать типы этих переменных на основе результатов анализа.

До TypeScript 4.4 анализ потока управления применялся только к коду внутри инструкции if, но начиная с TypeScript 4.4 он также может применяться к условным выражениям и доступу к дискриминирующим свойствам, на которые косвенно ссылаются через переменные const.

Например:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
```

```
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Несколько примеров, в которых сужение не происходит:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Ошибка: сужения нет, поскольку isString не
является const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
            obj.foo; // Ошибка: сужения нет, поскольку obj
присваивается в теле функции
    }
};
```

Примечание: в условных выражениях анализируется до пяти уровней косвенности.

Предикаты типов

Предикаты типов в TypeScript — это функции, которые возвращают логическое значение и используются для

сужения типа переменной до более конкретного типа.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 автоматически выводит предикаты типов (например, `x is T`) в таких функциях, как `.filter`, поэтому он понимает, когда отфильтровываются такие значения, как undefined. Это даёт более точные типы и меньше ошибок; такой вывод работает для однозначных проверок (например, `x !== undefined`), но не для неоднозначных, таких как `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Дискриминируемые объединения

Дискриминируемые объединения в TypeScript — это вид типа-объединения, который использует общее свойство, называемое дискриминантом, для сужения множества возможных типов объединения.

```
type Square = {
    kind: 'square'; // Дискриминант
    size: number;
};

type Circle = {
```

```
    kind: 'circle'; // Дискриминант
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Тип данных never

Когда тип переменной сужается до типа, который не может содержать никаких значений, компилятор TypeScript выводит, что переменная должна иметь тип `never`. Это связано с тем, что тип never представляет значение, которое никогда не может быть получено.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
          // Здесь val имеет тип never, поскольку он не может
быть ничем, кроме строки или числа
```

```
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Проверка исчерпываемости

Проверка исчерпываемости — это возможность TypeScript убедиться, что в инструкции `switch` или `if` обработаны все возможные варианты дискриминируемого объединения.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
            console.log(exhaustiveCheck); // Эта строка никогда не будет выполнена
    }
};
```

Тип `never` используется для обеспечения исчерпывающего охвата в ветви по умолчанию: TypeScript выдаст ошибку, если в тип Direction будет добавлено новое значение, которое не обрабатывается в инструкции switch.

Объектные типы

В TypeScript объектные типы описывают форму объекта. Они определяют имена и типы свойств объекта, а также указывают, являются ли эти свойства обязательными или необязательными.

В TypeScript объектные типы можно определить двумя основными способами:

Интерфейс определяет форму объекта, задавая имена, типы и необязательность его свойств.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Псевдоним типа, подобно интерфейсу, определяет форму объекта. Однако он также может создавать новый пользовательский тип на основе существующего типа или сочетания существующих типов. Это включает определение типов-объединений, типов-пересечений и других сложных типов.

```
type Point = {
    x: number;
    y: number;
};
```

Тип также можно определить анонимно:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Тип кортежа (анонимный)

Тип кортежа — это тип, представляющий массив с фиксированным количеством элементов и соответствующими им типами. Тип кортежа задаёт определённое количество элементов и их типы в фиксированном порядке. Типы кортежей полезны, когда необходимо представить набор значений определённых типов, где положение каждого элемента в массиве имеет конкретный смысл.

```
type Point = [number, number];
```

Именованный тип кортежа (с метками)

Типы кортежей могут включать необязательные метки или имена для каждого элемента. Эти метки улучшают читаемость и помогают инструментам разработки, но не влияют на операции, которые можно выполнять с кортежами.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Именованный кортеж вместе с анонимным кортежем
```

Кортеж фиксированной длины

Кортеж фиксированной длины — это особый тип кортежа, который задаёт фиксированное количество элементов определённых типов и запрещает изменять длину кортежа после его определения.

Кортежи фиксированной длины полезны, когда необходимо представить набор значений с определённым количеством элементов и определёнными типами и требуется гарантировать, что длина и типы кортежа не будут случайно изменены.

```
const x = [10, 'hello'] as const;
x.push(2); // Ошибка
```

Тип-объединение

Тип-объединение представляет значение, которое может принадлежать одному из нескольких типов. Типы-объединения обозначаются символом | между возможными типами.

```
let x: string | number;
x = 'hello'; // Допустимо
x = 123; // Допустимо
```

Типы-пересечения

Тип-пересечение представляет значение, которое обладает всеми свойствами двух или более типов. Типы-пересечения обозначаются символом & между типами.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Пересечение
```

```
const j: J = {
    a: 'a',
    b: 'b',
};
```

Индексирование типов

Индексирование типов означает возможность определять типы, индексируемые заранее неизвестным ключом. Для указания типа свойств, не объявленных явно, используется индексная сигнатура.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Возвращает a
```

Тип из значения

Тип из значения в TypeScript означает автоматический вывод типа из значения или выражения посредством механизма вывода типов.

```
const x = 'x'; // TypeScript выводит для 'x' строковый литеральный тип с 'const' (неизменяемый), но расширяет его до 'string' с 'let' (допускает переприсваивание).
```

Тип из возвращаемого значения функции

Тип из возвращаемого значения функции означает возможность автоматически вывести возвращаемый тип

функции на основе её реализации. Это позволяет TypeScript определить тип возвращаемого функцией значения без явных аннотаций типов.

```
const add = (x: number, y: number) => x + y; // TypeScript может вывести, что функция возвращает число
```

Тип из модуля

Тип из модуля — это возможность использовать экспортируемые значения модуля для автоматического вывода их типов. Когда модуль экспортирует значение определённого типа, TypeScript может использовать эту информацию, чтобы автоматически вывести тип данного значения при его импорте в другой модуль.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r имеет тип number
```

Сопоставляемые типы

Сопоставляемые типы в TypeScript позволяют создавать новые типы на основе существующего типа, преобразуя каждое свойство с помощью функции сопоставления. Сопоставляя существующие типы, можно создавать новые типы, представляющие ту же информацию в другом формате. Чтобы создать сопоставляемый тип, обращаются к свойствам существующего типа с помощью оператора `keyof`, а затем изменяют их для получения нового типа. В следующем примере:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

мы определяем MyMappedType, который выполняет сопоставление свойств T, создавая новый тип, где каждое свойство представляет собой массив исходного типа этого свойства. С его помощью мы создаём MyNewType, представляющий ту же информацию, что и MyType, но с каждым свойством в виде массива.

Модификаторы сопоставляемых типов

Модификаторы сопоставляемых типов в TypeScript позволяют преобразовывать свойства существующего типа:

- `readonly` или `+readonly`: делает свойство сопоставляемого типа доступным только для чтения.
- `-readonly`: делает свойство сопоставляемого типа изменяемым.
- `?`: обозначает свойство сопоставляемого типа как необязательное.

Примеры:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Все свойства помечены как доступные только для чтения

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Все свойства помечены как изменяемые

type MyPartial<T> = { [P in keyof T]?: T[P] }; // Все свойства помечены как необязательные
```

Условные типы

Условные типы позволяют создавать тип, зависящий от условия: создаваемый тип определяется результатом этого условия. Они задаются с помощью ключевого слова `extends` и тернарного оператора, который условно выбирает один из двух типов.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Тип true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Тип false
```

Дистрибутивные условные типы

Дистрибутивные условные типы — это возможность распределить тип по объединению типов, применяя преобразование отдельно к каждому члену объединения. Это может быть особенно полезно при работе с сопоставляемыми типами или типами высшего порядка.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
```

```
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

Вывод типа с помощью infer в условных типах

Ключевое слово `infer` используется в условных типах для вывода (извлечения) типа обобщённого параметра из зависящего от него типа. Это позволяет создавать более гибкие определения типов, пригодные для повторного использования.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Предопределённые условные типы

В TypeScript предопределённые условные типы — это встроенные условные типы, предоставляемые языком. Они предназначены для выполнения распространённых преобразований типов с учётом характеристик заданного типа.

`Exclude<UnionType, ExcludedType>`: удаляет из Type все типы, которые могут быть присвоены ExcludedType.

`Extract<Type, Union>`: извлекает из Union все типы, которые могут быть присвоены Type.

`NonNullable<Type>`: удаляет null и undefined из Type.

`ReturnType<Type>`: извлекает тип возвращаемого значения функции Type.

`Parameters<Type>`: извлекает типы параметров функции Type.

`Required<Type>`: делает все свойства Type обязательными.

`Partial<Type>`: делает все свойства Type необязательными.

`Readonly<Type>`: делает все свойства Type доступными только для чтения.

Шаблонные типы объединений

Шаблонные типы объединений можно использовать для объединения текста и работы с ним внутри системы типов, например:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Тип any

Тип `any` — это специальный тип (универсальный супертип), который может представлять значение любого типа (примитивы, объекты, массивы, функции, ошибки, символы). Он часто используется в ситуациях, когда тип значения неизвестен во время компиляции, или при работе со значениями из внешних API либо библиотек, не имеющих типизации TypeScript.

Используя тип `any`, вы указываете компилятору TypeScript, что значения должны рассматриваться без каких-либо ограничений. Чтобы обеспечить максимальную типобезопасность кода, учитывайте следующее:

- Ограничивайте использование any конкретными случаями, когда тип действительно неизвестен.
- Не возвращайте из функции значения типа any, поскольку это снижает типобезопасность использующего их кода.
- Если необходимо подавить сообщение компилятора, вместо any используйте `@ts-ignore`.

```
let value: any;
value = true; // Допустимо
value = 7; // Допустимо
```

Тип unknown

В TypeScript тип unknown представляет значение неизвестного типа. В отличие от типа any, допускающего значение любого типа, unknown требует проверки или утверждения типа, прежде чем значение можно будет использовать определённым образом. Поэтому никакие операции со значением unknown не разрешены без предварительного утверждения или сужения до более конкретного типа.

Тип unknown можно присвоить только типам any и unknown; это типобезопасная альтернатива any.

```
let value: unknown;

let value1: unknown = value; // Допустимо
let value2: any = value; // Допустимо
let value3: boolean = value; // Недопустимо
let value4: number = value; // Недопустимо
```

```
const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Тип void

Тип `void` используется, чтобы указать, что функция не возвращает значения.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Тип never

Тип `never` представляет значения, которые никогда не возникают. Он используется для обозначения функций или выражений, которые никогда не возвращают управление либо выбрасывают ошибку.

Например, бесконечный цикл:

```
const infiniteLoop = (): never => {
    while (true) {
        // выполнить действие
    }
};
```

Выбрасывание ошибки:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Тип `never` полезен для обеспечения типобезопасности и выявления потенциальных ошибок в коде. Он помогает TypeScript анализировать и выводить более точные типы при использовании совместно с другими типами и операторами управления потоком выполнения, например:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // двигаться вверх
            break;
        case 'down':
            // двигаться вниз
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw  new  Error(`Unhandled  direction:
${exhaustiveCheck}`);
    }
};
```

Интерфейс и тип

Общий синтаксис

В TypeScript интерфейсы определяют структуру объектов, задавая имена и типы свойств или методов, которые должен иметь объект. Общий синтаксис определения интерфейса в TypeScript выглядит следующим образом:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
```

```
    // ...
}
```

Аналогично для определения типа:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` или `type TypeName`: определяет имя интерфейса. `property1: Type1`: задаёт свойства интерфейса вместе с соответствующими типами. Можно определить несколько свойств, разделяя их точкой с запятой. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: задаёт методы интерфейса. Метод определяется его именем, за которым следует список параметров в круглых скобках и тип возвращаемого значения. Можно определить несколько методов, разделяя их точкой с запятой.

Пример интерфейса:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Пример типа:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

В TypeScript типы используются для определения структуры данных и обеспечения проверки типов. Существует несколько распространённых вариантов синтаксиса для определения типов в TypeScript в зависимости от конкретного сценария использования. Ниже приведены некоторые примеры:

Базовые типы

```
let myNumber: number = 123; // тип number
let myBoolean: boolean = true; // тип boolean
let myArray: string[] = ['a', 'b']; // массив строк
let myTuple: [string, number] = ['a', 123]; // кортеж
```

Объекты и интерфейсы

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Типы объединения и пересечения

```
type MyType = string | number; // Тип объединения
let myUnion: MyType = 'hello'; // Может быть строкой
myUnion = 123; // Или числом

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Тип пересечения
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Объект со свойствами name и age
```

Встроенные примитивные типы

В TypeScript есть несколько встроенных примитивных типов, которые можно использовать для определения

переменных, параметров функций и типов возвращаемых значений:

- `number`: представляет числовые значения, включая целые числа и числа с плавающей точкой.
- `string`: представляет текстовые данные
- `boolean`: представляет логические значения, которые могут быть либо true, либо false.
- `null`: представляет отсутствие значения.
- `undefined`: представляет значение, которое не было присвоено или определено.
- `symbol`: представляет уникальный идентификатор. Символы обычно используются в качестве ключей свойств объектов.
- `bigint`: представляет целые числа произвольной точности.
- `any`: представляет динамический или неизвестный тип. Переменные типа any могут содержать значения любого типа и обходят проверку типов.
- `void`: представляет отсутствие какого-либо типа. Обычно используется как тип возвращаемого значения функций, которые не возвращают значение.
- `never`: представляет тип значений, которые никогда не возникают. Обычно используется как тип возвращаемого значения функций, которые выбрасывают ошибку или входят в бесконечный цикл.

Распространённые встроенные объекты JS

TypeScript является надмножеством JavaScript и включает все широко используемые встроенные объекты JavaScript.

Подробный список этих объектов можно найти в документации Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Ниже приведены некоторые широко используемые встроенные объекты JavaScript:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Перегрузки

Перегрузки функций в TypeScript позволяют определить несколько сигнатур для одного имени функции, благодаря чему функцию можно вызывать несколькими способами. Рассмотрим пример:

```
// Перегрузки
function sayHi(name: string): string;
```

```
function sayHi(names: string[]): string[];

// Реализация
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Допустимо
sayHi(['aa', 'bb']); // Допустимо
```

Ещё один пример использования перегрузок функций внутри `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // перегрузка
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // реализация
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
```

```
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Слияние и расширение

Слияние и расширение — это два разных понятия, связанных с работой с типами и интерфейсами.

Слияние позволяет объединить несколько объявлений с одним именем в единое определение, например когда интерфейс с одним именем определяется несколько раз:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Расширение — это возможность расширять существующие типы или интерфейсы либо наследовать от них для создания новых. Этот механизм позволяет добавлять дополнительные свойства или методы к существующему типу без изменения его исходного определения. Пример:

```
interface Animal {
    name: string;
    eat(): void;
}
```

```
interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Различия между типом и интерфейсом

Слияние объявлений (дополнение):

Интерфейсы поддерживают слияние объявлений: можно определить несколько интерфейсов с одинаковым именем, и TypeScript объединит их в один интерфейс с совокупностью их свойств и методов. Типы, напротив, не поддерживают слияние объявлений. Это может быть полезно, если требуется добавить функциональность или настроить существующие типы без изменения исходных определений, а также дополнить отсутствующие либо исправить неверные типы.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
```

```
    x: 'xx',
    y: 'yy',
};
```

Расширение других типов и интерфейсов:

И типы, и интерфейсы могут расширять другие типы и интерфейсы, но синтаксис различается. Для интерфейсов используется ключевое слово `extends`, позволяющее наследовать свойства и методы других интерфейсов. Однако интерфейс не может расширять сложный тип, например тип объединения.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Для типов оператор & используется для объединения нескольких типов в один тип (пересечение).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};
```

```
const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Типы объединения и пересечения:

Типы предоставляют больше возможностей при определении типов объединения и пересечения. С помощью ключевого слова `type` можно легко создавать типы объединения, используя оператор `|`, и типы пересечения, используя оператор `&`. Хотя интерфейсы также могут косвенно представлять типы объединения, у них нет встроенной поддержки типов пересечения.

```
type Department = 'dep-x' | 'dep-y'; // Объединение

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Пересечение
```

Пример с интерфейсами:

```
interface A {
    x: 'x';
}
interface B {
```

```
    y: 'y';
}

type C = A | B; // Объединение интерфейсов
```

Класс

Общий синтаксис класса

Ключевое слово `class` используется в TypeScript для определения класса. Ниже приведён пример:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Ключевое слово `class` используется для определения класса с именем “Person”.

Класс имеет два закрытых свойства: name с типом `string` и age с типом `number`.

Конструктор определяется с помощью ключевого слова `constructor`. Он принимает name и age в качестве параметров и присваивает их соответствующим свойствам.

Класс имеет `public`-метод sayHi, который выводит приветственное сообщение в консоль.

Чтобы создать экземпляр класса в TypeScript, можно использовать ключевое слово `new`, после которого следуют имя класса и круглые скобки `()`. Например:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Вывод: Hello, my name is John Doe and I am
25 years old.
```

Конструктор

Конструкторы — это специальные методы класса, которые используются для инициализации свойств объекта при создании экземпляра класса.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Конструктор можно перегрузить, используя следующий синтаксис:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

В TypeScript можно определить несколько перегрузок конструктора, однако реализация может быть только одна и должна быть совместима со всеми перегрузками. Этого можно добиться с помощью необязательного параметра.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }
```

```
    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Закрытые и защищённые конструкторы

В TypeScript конструкторы можно объявить закрытыми или защищёнными, что ограничивает их доступность и использование.

Закрытые конструкторы: Могут вызываться только внутри самого класса. Закрытые конструкторы часто используются, когда необходимо реализовать паттерн «Одиночка» или разрешить создание экземпляров только фабричному методу внутри класса.

Защищённые конструкторы: Защищённые конструкторы полезны, когда необходимо создать базовый класс, экземпляры которого не следует создавать напрямую, но от которого могут наследоваться подклассы.

```
class BaseClass {
    protected constructor() {}
}
```

```
class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Попытка напрямую создать экземпляр базового класса приведёт к ошибке
// const baseObj = new BaseClass(); // Ошибка: конструктор класса 'BaseClass' защищён.

// Создание экземпляра производного класса
const derivedObj = new DerivedClass(10);
```

Модификаторы доступа

Модификаторы доступа `private`, `protected` и `public` используются для управления видимостью и доступностью членов классов TypeScript, таких как свойства и методы. Эти модификаторы необходимы для обеспечения инкапсуляции и установления границ доступа к внутреннему состоянию класса и его изменению.

Модификатор `private` разрешает доступ к члену класса только внутри содержащего его класса.

Модификатор `protected` разрешает доступ к члену класса внутри содержащего его класса и производных классов.

Модификатор `public` предоставляет неограниченный доступ к члену класса, позволяя обращаться к нему откуда угодно.

Геттеры и сеттеры

Геттеры и сеттеры — это специальные методы, позволяющие определять пользовательское поведение при доступе к свойствам класса и их изменении. Они позволяют инкапсулировать внутреннее состояние объекта и добавлять логику при получении или установке значений свойств. В TypeScript геттеры и сеттеры определяются с помощью ключевых слов `get` и `set` соответственно. Рассмотрим пример:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Автоаксессоры в классах

В TypeScript версии 4.9 добавлена поддержка автоаксессоров — планируемой возможности ECMAScript. Они похожи на свойства класса, но объявляются с помощью ключевого слова “accessor”.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
```

```
        this.name = name;
    }
}
```

Автоаксессоры преобразуются в закрытые аксессоры `get` и `set`, работающие с недоступным извне свойством.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

В TypeScript ключевое слово `this` внутри методов или конструкторов класса ссылается на текущий экземпляр класса. Оно позволяет получать доступ к свойствам и методам класса и изменять их из области видимости самого класса. Это даёт возможность получать доступ к внутреннему состоянию объекта и управлять им в его собственных методах.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
```

```
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Свойства-параметры

Свойства-параметры позволяют объявлять и инициализировать свойства класса непосредственно в параметрах конструктора, избегая повторяющегося шаблонного кода. Например:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // Ключевые слова "private" и "public" в конструкторе
                // автоматически объявляют и инициализируют
соответствующие свойства класса.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Абстрактные классы

Абстрактные классы используются в TypeScript главным образом для наследования. Они позволяют определять общие свойства и методы, которые могут наследоваться подклассами. Это полезно, когда требуется определить общее поведение и обязать подклассы реализовать определённые методы. Абстрактные классы позволяют создать иерархию классов, в которой абстрактный базовый класс предоставляет подклассам общий интерфейс и общую функциональность.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Вывод: Whiskers meows.
```

С обобщёнными типами

Классы с обобщёнными типами позволяют определять классы, пригодные для повторного использования и способные работать с разными типами.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Декораторы

Декораторы предоставляют механизм для добавления метаданных, изменения поведения, проверки или расширения функциональности целевого элемента. Это функции, выполняемые во время работы программы. К одному объявлению можно применить несколько декораторов.

Декораторы являются экспериментальной возможностью, а следующие примеры совместимы только с TypeScript версии 5 или выше при использовании ES6.

В версиях TypeScript до 5 их необходимо включить с помощью свойства `experimentalDecorators` в файле `tsconfig.json` или параметра `--experimentalDecorators` в командной строке (но следующий пример работать не будет).

Некоторые распространённые сценарии использования декораторов:

- Отслеживание изменений свойств.
- Отслеживание вызовов методов.
- Добавление дополнительных свойств или методов.
- Проверка во время выполнения.
- Автоматическая сериализация и десериализация.
- Логирование.
- Авторизация и аутентификация.
- Защита от ошибок.

Примечание: декораторы в версии 5 не позволяют декорировать параметры.

Типы декораторов:

Декораторы классов

Декораторы классов полезны для расширения существующего класса, например для добавления свойств или методов либо сбора экземпляров класса. В следующем примере мы добавляем метод `toString`, преобразующий класс в строковое представление.

```
type Constructor<T = {}> = new (...args: any[]) => T;
```

```
function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Вывод:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Декоратор свойства

Декораторы свойств полезны для изменения поведения свойства, например для изменения начальных значений. В следующем коде приведён скрипт, который всегда задаёт значение свойства в верхнем регистре:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Вывод: HELLO!
```

Декоратор метода

Декораторы методов позволяют изменять или расширять поведение методов. Ниже приведён пример простого логгера:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
```

```
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Будет выведено:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Декораторы геттеров и сеттеров

Декораторы геттеров и сеттеров позволяют изменять или расширять поведение аксессоров класса. Они полезны, например, для проверки значений, присваиваемых свойствам. Ниже приведён простой пример декоратора геттера:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
```

```
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Допустимо: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Исключение: Invalid!
```

Метаданные декораторов

Метаданные декораторов упрощают применение и использование метаданных в любом классе. Декораторы могут обращаться к новому свойству metadata объекта контекста, которое может служить ключом как для

примитивов, так и для объектов. Доступ к метаданным класса можно получить через `Symbol.metadata`.

Метаданные можно использовать для различных целей, таких как отладка, сериализация или внедрение зависимостей с помощью декораторов.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Простой полифил

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
        | ClassMethodDecoratorContext; // Контекст содержит метаданные свойства: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Задать примитивное значение в объекте метаданных
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Получение метаданных

console.log(JSON.stringify(metadata)); //
{"bar":true,"baz":true,"foo":true}
```

Наследование

Наследование — это механизм, благодаря которому класс может наследовать свойства и методы другого класса, называемого базовым классом или суперклассом. Производный класс, также называемый дочерним классом или подклассом, может расширять и специализировать функциональность базового класса, добавляя новые свойства и методы либо переопределяя существующие.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Создание экземпляра базового класса
```

```
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Создание экземпляра производного класса
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript не поддерживает множественное наследование в традиционном смысле и вместо этого допускает наследование от одного базового класса. TypeScript поддерживает использование нескольких интерфейсов. Интерфейс может определять контракт структуры объекта, а класс может реализовывать несколько интерфейсов. Благодаря этому класс может наследовать поведение и структуру из нескольких источников.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Ключевое слово `class` в TypeScript, как и в JavaScript, часто называют синтаксическим сахаром. Оно было введено в ECMAScript 2015 (ES6), чтобы предоставить более привычный синтаксис для создания объектов и работы с ними в стиле классов. Однако важно отметить, что TypeScript, являясь надмножеством JavaScript, в конечном счёте компилируется в JavaScript, который по своей сути остаётся прототипно-ориентированным.

Статические члены

TypeScript поддерживает статические члены. Для доступа к статическим членам класса можно указать имя класса и поставить после него точку, не создавая объект.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Инициализация свойств

В TypeScript существует несколько способов инициализировать свойства класса:

Непосредственно в объявлении:

В следующем примере эти начальные значения будут использованы при создании экземпляра класса.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

В конструкторе:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

С помощью параметров конструктора:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // Нет необходимости явно присваивать значения
свойствам.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Перегрузка методов

Перегрузка методов позволяет классу иметь несколько методов с одинаковым именем, но с разными типами или количеством параметров. Это позволяет вызывать метод разными способами в зависимости от переданных аргументов.

```
class MyClass {
    add(a: number, b: number): number; // Сигнатура перегрузки
1
    add(a: string, b: string): string; // Сигнатура перегрузки
2

     add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Выводит 15
```

Обобщения

Обобщения позволяют создавать переиспользуемые компоненты и функции, способные работать с несколькими типами. С помощью обобщений можно параметризовать типы, функции и интерфейсы, позволяя им работать с разными типами без их предварительного явного указания.

Обобщения делают код более гибким и пригодным для повторного использования.

Обобщённый тип

Чтобы определить обобщённый тип, параметры типа указывают в угловых скобках (<>), например:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Обобщённые классы

Обобщения также можно применять к классам, чтобы с помощью параметров типа они могли работать с несколькими типами. Это полезно для создания переиспользуемых определений классов, способных работать с различными типами данных и при этом сохранять типобезопасность.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
```

```
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Ограничения обобщений

Обобщённые параметры можно ограничивать с помощью ключевого слова `extends`, за которым следует тип или интерфейс, требованиям которого должен соответствовать параметр типа.

В следующем примере тип `T` должен иметь свойство `length` с корректно заданным типом, чтобы значение было допустимым:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Недопустимо
```

Примечательная возможность обобщений, появившаяся в версии 3.4 RC, — вывод типов для функций высшего порядка, который распространяет аргументы обобщённых типов:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
```

```
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Эта возможность упрощает типобезопасное программирование в бесточечном стиле, распространённом в функциональном программировании.

Контекстное сужение обобщённых типов

Контекстное сужение обобщённых типов — это механизм TypeScript, позволяющий компилятору сужать тип обобщённого параметра на основе контекста его использования. Он полезен при работе с обобщёнными типами в условных операторах:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Значение сужено до типа 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Значение сужено до типа 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Стираемые структурные типы

В TypeScript объекты не обязаны точно соответствовать определённому типу. Например, если создать объект, который удовлетворяет требованиям интерфейса, его можно использовать там, где требуется этот интерфейс, даже если между ними не была явно установлена связь. Пример:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Допустимо
```

Пространства имён

В TypeScript пространства имён используются для организации кода в логические контейнеры, предотвращая конфликты имён и позволяя объединять связанный код. Ключевое слово `export` позволяет обращаться к пространству имён извне модулей.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
```

```
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Символы

Символы — это примитивный тип данных, представляющий неизменяемое значение, глобальная уникальность которого гарантируется на протяжении всего времени выполнения программы.

Символы можно использовать в качестве ключей свойств объектов; кроме того, они позволяют создавать неперечисляемые свойства.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Теперь символы можно использовать в качестве ключей в WeakMap и WeakSet.

Директивы с тройной косой чертой

Директивы с тройной косой чертой — это специальные комментарии, содержащие инструкции для компилятора по обработке файла. Они начинаются с трёх последовательных косых черт (`///`), обычно размещаются в начале файла TypeScript и не влияют на поведение во время выполнения.

Директивы с тройной косой чертой используются для ссылки на внешние зависимости, указания поведения загрузки модулей, включения или отключения некоторых возможностей компилятора и других целей. Несколько примеров:

Ссылка на файл объявлений:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Указание формата модуля:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Включение параметров компилятора, в следующем примере — строгого режима:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Манипуляции с типами

Создание типов из типов

Новые типы можно создавать путём композиции, изменения или преобразования существующих типов.

Типы-пересечения (`&`):

Позволяют объединить несколько типов в один:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Пересечение A и B
const obj: C = { foo: 42, bar: 'hello' };
```

Типы-объединения (|):

Позволяют определить тип, который может быть одним из нескольких типов:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Сопоставляемые типы:

Позволяют преобразовать свойства существующего типа для создания нового типа:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Свойства становятся
доступными только для чтения
```

Условные типы:

Позволяют создавать типы на основе определённых условий:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Типы индексированного доступа

В TypeScript можно обращаться к типам свойств другого типа и манипулировать ими с помощью индекса `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Служебные типы

Для манипуляций с типами можно использовать несколько встроенных служебных типов. Ниже перечислены наиболее часто используемые:

Awaited<T>

Создаёт тип, рекурсивно разворачивающий типы Promise.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Создаёт тип, в котором все свойства T являются необязательными.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

Создаёт тип, в котором все свойства T являются обязательными.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Создаёт тип, в котором все свойства T доступны только для чтения.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;
```

```ts
const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Недопустимо
```

Record<K, T>

Создаёт тип с набором свойств K типа T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Создаёт тип, выбирая указанные свойства K из T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Создаёт тип, исключая указанные свойства K из T.

```ts
type Product = {
    name: string;
```

```
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Создаёт тип, исключая из T все значения типа U.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Создаёт тип, извлекая из T все значения типа U.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Создаёт тип, исключая null и undefined из T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Извлекает типы параметров функции типа T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Извлекает типы параметров функции-конструктора типа T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Извлекает тип возвращаемого значения функции типа T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Извлекает тип экземпляра из типа класса T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}
```

```
type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Извлекает тип параметра ‘this’ функции типа T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type  PersonThisType  =  ThisParameterType<Person['greet']>;  //
Person
```

OmitThisParameter<T>

Удаляет параметр ‘this’ из типа функции T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; //
() => string
```

ThisType<T>

Служит маркером контекстного типа `this`.

```
type Logger = {
    log: (error: string) => void;
};
```

```
let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
          this.log('some error'); // Допустимо, поскольку "log"
является частью "this".
        this.update(); // Недопустимо
    },
};
```

Uppercase<T>

Преобразует имя входного типа T в верхний регистр.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Преобразует имя входного типа T в нижний регистр.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Преобразует первую букву имени входного типа T в верхний регистр.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Преобразует первую букву имени входного типа T в нижний регистр.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer — это служебный тип, предназначенный для блокировки автоматического вывода типов в области действия обобщённой функции.

Пример:

```
// Автоматический вывод типов в области видимости обобщённой функции.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Здесь тип ("a" | "b" | "c")[]
```

С NoInfer:

```
// Пример функции, использующей NoInfer для предотвращения вывода типов
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Ошибка: аргумент типа '"c"' нельзя присвоить параметру типа '"a" | "b"'.
```

Прочее

Ошибки и обработка исключений

TypeScript позволяет перехватывать и обрабатывать ошибки с помощью стандартных механизмов обработки ошибок JavaScript:

Блоки Try-Catch-Finally:

```
try {
    // Код, который может вызвать исключение
} catch (error) {
    // Обработка ошибки
} finally {
        // Код, который выполняется всегда; блок finally
необязателен
}
```

Также можно обрабатывать ошибки разных типов:

```
try {
    // Код, который может вызывать ошибки разных типов
} catch (error) {
    if (error instanceof TypeError) {
        // Обработка TypeError
    } else if (error instanceof RangeError) {
        // Обработка RangeError
    } else {
        // Обработка других ошибок
    }
}
```

Пользовательские типы ошибок:

Более конкретные ошибки можно определять, расширяя класс Error с помощью `class`:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Классы-примеси

Классы-примеси позволяют объединять и компоновать поведение нескольких классов в одном классе. Они дают возможность повторно использовать и расширять функциональность без необходимости создавать глубокие цепочки наследования.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Расширить MyClass, включив поведение Identifiable и
Selectable
interface MyClass extends Identifiable, Selectable {}

// Функция для применения примесей к классу
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
```

```
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Применить примеси к MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Асинхронные возможности языка

Поскольку TypeScript является надмножеством JavaScript, он имеет встроенные асинхронные возможности JavaScript, такие как:

Промисы:

Промисы позволяют обрабатывать асинхронные операции и их результаты с помощью таких методов, как `.then()` и `.catch()`, для обработки успешного выполнения и ошибок.

Подробнее: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Ключевые слова async/await предоставляют синтаксис для работы с Promise, внешне напоминающий синхронный код. Ключевое слово `async` используется для определения асинхронной функции, а `await` — внутри асинхронной функции, чтобы приостановить выполнение до успешного завершения или отклонения Promise.

Подробнее: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

В TypeScript хорошо поддерживаются следующие API:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Итераторы и генераторы

И итераторы, и генераторы хорошо поддерживаются в TypeScript.

Итераторы — это объекты, реализующие протокол итератора и предоставляющие доступ к элементам коллекции или последовательности по одному. Итератор

представляет собой структуру, содержащую указатель на следующий элемент итерации. У итераторов есть метод `next()`, который возвращает следующее значение последовательности вместе с логическим значением `done`, указывающим, завершена ли последовательность.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Генераторы — это специальные функции, определяемые с помощью синтаксиса `function*`, который упрощает создание итераторов. Они используют ключевое слово `yield` для определения последовательности значений и автоматически приостанавливают и возобновляют выполнение при запросе значений.

Генераторы упрощают создание итераторов и особенно полезны при работе с большими или бесконечными последовательностями.

Пример:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript также поддерживает асинхронные итераторы и асинхронные генераторы.

Подробнее:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Справочник по TsDocs JSDoc

При работе с кодовой базой JavaScript можно помочь TypeScript вывести правильный тип с помощью комментариев JSDoc с дополнительной аннотацией, предоставляющей информацию о типах.

Пример:

```
/**
 * Вычисляет степень заданного числа
 * @constructor
 * @param {number} base - Основание выражения
 * @param {number} exponent - Показатель степени выражения
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Полная документация доступна по этой ссылке: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Начиная с версии 3.7, определения типов из файлов `.d.ts` можно генерировать из синтаксиса JSDoc в JavaScript. Дополнительная информация доступна здесь: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Для пакетов в организации @types используется специальное соглашение об именовании, предназначенное для предоставления определений типов существующих библиотек или модулей JavaScript. Например, команда:

```
npm install --save-dev @types/lodash
```

установит определения типов для `lodash` в текущий проект.

Чтобы внести вклад в определения типов пакета `@types`, отправьте пул-реквест в репозиторий https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) — это расширение синтаксиса языка JavaScript, позволяющее писать HTML-подобный код в файлах JavaScript или TypeScript. Обычно оно используется в React для определения структуры HTML.

TypeScript расширяет возможности JSX, предоставляя проверку типов и статический анализ.

Чтобы использовать JSX, необходимо задать параметр компилятора `jsx` в файле `tsconfig.json`. Два распространённых варианта конфигурации:

- “preserve”: создаёт файлы .jsx, оставляя JSX без изменений. Этот вариант предписывает TypeScript сохранять синтаксис JSX как есть и не преобразовывать его в процессе компиляции. Его можно использовать, если у вас есть отдельный инструмент, например Babel, который выполняет преобразование.

- “react”: включает встроенное в TypeScript преобразование JSX. Будет использоваться React.createElement.

Все варианты доступны здесь: https://www.typescriptlang.org/tsconfig#jsx

Модули ES6

TypeScript поддерживает ES6 (ECMAScript 2015) и многие последующие версии. Это означает, что вы можете использовать синтаксис ES6, включая стрелочные функции, шаблонные литералы, классы, модули, деструктуризацию и многое другое.

Чтобы включить возможности ES6 в проекте, можно указать свойство `target` в tsconfig.json.

Пример конфигурации:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Оператор возведения в степень ES7

Оператор возведения в степень (**) вычисляет значение, полученное возведением первого операнда в степень

второго операнда. Он работает аналогично `Math.pow()`, но дополнительно может принимать BigInt в качестве операндов. TypeScript полностью поддерживает этот оператор, если в файле tsconfig.json параметру `target` задано значение `es2016` или более поздней версии.

```
console.log(2 ** (2 ** 2)); // 16
```

Инструкция for-await-of

Это полностью поддерживаемая в TypeScript возможность JavaScript, которая позволяет перебирать асинхронные итерируемые объекты при целевой версии `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Новое метасвойство target

В TypeScript можно использовать метасвойство `new.target`, которое позволяет определить, использовался ли оператор new при вызове функции или конструктора. Оно позволяет выяснить, был ли объект создан в результате вызова конструктора.

```
class Parent {
    constructor() {
                console.log(new.target);  // Выводит функцию-
конструктор, использованную для создания экземпляра
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Динамические выражения импорта

Модули можно загружать условно или отложенно по требованию с помощью предложения ECMAScript о динамическом импорте, которое поддерживается в TypeScript.

Синтаксис динамических выражений импорта в TypeScript выглядит следующим образом:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
                const  widget  =  await  import('./widget');  //
Динамический импорт
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Эта команда запускает компилятор TypeScript с параметром `--watch`, обеспечивая автоматическую перекомпиляцию файлов TypeScript при каждом их изменении.

```
tsc --watch
```

Начиная с TypeScript версии 4.9, отслеживание файлов в основном опирается на события файловой системы и автоматически переключается на опрос, если установить наблюдатель на основе событий невозможно.

Оператор утверждения о ненулевом значении

Оператор утверждения о ненулевом значении (постфиксный !), также называемый утверждением об определённом присваивании, — это возможность TypeScript, позволяющая утверждать, что переменная или свойство не имеют значения null или undefined, даже если статический анализ типов TypeScript предполагает обратное. С помощью этой возможности можно исключить любые явные проверки.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Объявления со значениями по умолчанию

Объявления со значениями по умолчанию используются, когда переменной или параметру присваивается значение по умолчанию. Это означает, что, если значение для этой переменной или параметра не указано, будет использовано значение по умолчанию.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Опциональная цепочка

Оператор опциональной цепочки `?.` работает подобно обычному оператору точки (`.`), используемому для доступа к свойствам или методам. Однако он корректно обрабатывает значения null и undefined: прекращает вычисление выражения и возвращает `undefined` вместо выброса ошибки.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Оператор объединения с null

Оператор объединения с null `??` возвращает значение справа, если значение слева равно `null` или `undefined`; в противном случае он возвращает значение слева.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Типы шаблонных литералов

Типы шаблонных литералов позволяют манипулировать строковыми значениями на уровне типов и создавать новые строковые типы на основе существующих. Они полезны для создания более выразительных и точных типов на основе строковых операций.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Перегрузка функций

Перегрузка функций позволяет определить несколько сигнатур для одного имени функции, каждая с разными типами параметров и возвращаемыми типами. При вызове перегруженной функции TypeScript использует

переданные аргументы, чтобы определить подходящую сигнатуру функции:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Рекурсивные типы

Рекурсивный тип — это тип, который может ссылаться на самого себя. Это полезно для определения структур данных с иерархической или рекурсивной структурой (с потенциально бесконечной вложенностью), таких как связные списки, деревья и графы.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Рекурсивные условные типы

В TypeScript можно определять сложные отношения между типами с помощью логики и рекурсии. Разберём это простыми словами:

Условные типы позволяют определять типы на основе логических условий:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Рекурсия означает, что определение типа ссылается на самого себя внутри собственного определения:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Рекурсивные условные типы объединяют условную логику и рекурсию. Это означает, что определение типа может зависеть от самого себя через условную логику, создавая сложные и гибкие отношения между типами.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Поддержка модулей ECMAScript в Node

Node.js добавил поддержку модулей ECMAScript начиная с версии 15.3.0, а TypeScript поддерживает модули

ECMAScript в Node.js с версии 4.7. Эту поддержку можно включить, присвоив свойству `module` значение `nodenext` в файле tsconfig.json. Пример:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js поддерживает два расширения файлов модулей: `.mjs` для модулей ES и `.cjs` для модулей CommonJS. Соответствующие расширения файлов в TypeScript — `.mts` для модулей ES и `.cts` для модулей CommonJS. Когда компилятор TypeScript транспилирует эти файлы в JavaScript, он создаёт файлы `.mjs` и `.cjs`.

Если вы хотите использовать модули ES в своём проекте, можно присвоить свойству `type` значение “module” в файле package.json. Это указывает Node.js рассматривать проект как проект с модулями ES.

Кроме того, TypeScript поддерживает объявления типов в файлах `.d.ts`. Эти файлы объявлений предоставляют информацию о типах для библиотек или модулей, написанных на TypeScript, позволяя другим разработчикам использовать их вместе с проверкой типов и автодополнением TypeScript.

Функции утверждения

В TypeScript функции утверждения — это функции, которые указывают на проверку определённого условия на основе своего возвращаемого значения. В простейшей форме функция утверждения проверяет переданный предикат и выбрасывает ошибку, если предикат имеет значение false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Или её можно объявить как функциональное выражение:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Функции утверждения похожи на защитники типов. Изначально защитники типов были введены для выполнения проверок во время выполнения и гарантии типа значения в определённой области видимости. В частности, защитник типа — это функция, которая вычисляет предикат типа и возвращает логическое значение, указывающее, является ли предикат истинным или ложным. Это немного отличается от функций утверждения, предназначенных для выброса ошибки вместо возврата false, когда предикат не выполняется.

Пример защитника типа:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Вариативные кортежные типы

Вариативные кортежные типы — это возможность, представленная в TypeScript версии 4.0, поэтому для начала вспомним, что такое кортеж:

Кортежный тип — это массив с определённой длиной, тип каждого элемента которого известен:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Термин «вариативный» означает неопределённую арность (приём переменного количества аргументов).

Вариативный кортеж — это кортежный тип, обладающий всеми прежними свойствами, но его точная форма ещё не определена:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

В предыдущем коде видно, что форма кортежа определяется переданным обобщённым типом `T`.

Вариативные кортежи могут принимать несколько обобщённых типов, что делает их очень гибкими:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Новые вариативные кортежи позволяют использовать:

- Операторы расширения в синтаксисе кортежных типов теперь могут быть обобщёнными, поэтому мы можем представлять операции высшего порядка над кортежами и массивами, даже если не знаем фактических типов, с которыми работаем.
- Остаточные элементы могут находиться в любом месте кортежа.

Пример:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Объектные типы-обёртки

Объектные типы-обёртки — это объекты-обёртки, которые используются для представления примитивных типов в виде объектов. Эти объекты-обёртки предоставляют

дополнительные возможности и методы, недоступные непосредственно для примитивных значений.

Когда вы обращаетесь к методу вроде `charAt` или `normalize` у примитивного значения `string`, JavaScript оборачивает его в объект `String`, вызывает метод, а затем отбрасывает объект.

Демонстрация:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript представляет это различие, предоставляя отдельные типы для примитивов и соответствующих им объектов-обёрток:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Объектные типы-обёртки обычно не нужны. Избегайте их использования и вместо них применяйте примитивные типы, например `string` вместо `String`.

Ковариантность и контравариантность в TypeScript

Ковариантность и контравариантность описывают отношения типов внутри обобщённых типов. `Dog` является

`Animal`, но не каждый `Animal` является `Dog`.

Ковариантность (возврат значений): функция, возвращающая `Dog`, может заменить функцию, возвращающую `Animal`. Любая собака — животное, но функция, возвращающая произвольное животное, может вернуть кошку.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Допустимо
produceDog = produceAnimal; // Ошибка
```

Контравариантность (приём значений): функция, принимающая любой `Animal`, может заменить функцию, принимающую `Dog`. Она обработает и собаку; функция только для собак не может безопасно обработать любое животное.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;
```

```
consumeDog = consumeAnimal; // Допустимо
consumeAnimal = consumeDog; // Ошибка при включённом strictFunctionTypes
```

Стрелки показывают направление безопасного присваивания:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Изменяемые массивы: TypeScript также разрешает присвоить `Dog[]` переменной типа `Animal[]`. Обе переменные ссылаются на один массив; через более общий тип можно добавить `Animal`, который не является собакой.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Разрешено, но небезопасно
// Теперь dogs содержит Animal, который не является Dog.
```

Настройка компилятора: при `strictFunctionTypes` параметры обычных функций проверяются контравариантно. Без этого параметра TypeScript может допускать оба направления (бивариантность), даже если это небезопасно. Параметры объявлений методов и конструкторов — исключение.

Необязательные аннотации вариантности для параметров типов

Начиная с TypeScript 4.7, `out` обозначает ковариантность, `in` — контравариантность, а `in out` — инвариантность. Обычно

TypeScript выводит вариантность сам. Аннотации должны соответствовать использованию обобщённого параметра, а не произвольно менять его поведение.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Индексные сигнатуры с шаблонами строковых литералов

Индексные сигнатуры с шаблонами строковых литералов позволяют определять гибкие индексные сигнатуры с помощью шаблонов строковых литералов. Эта возможность позволяет создавать объекты, к которым можно обращаться по строковым ключам, соответствующим определённым шаблонам, обеспечивая более точный контроль при доступе к свойствам и манипуляциях с ними.

Начиная с версии 4.4, TypeScript позволяет использовать индексные сигнатуры для символов и шаблонов строковых литералов.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
```

```
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Ключ уникального символа
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

Оператор satisfies

Оператор `satisfies` позволяет проверить, удовлетворяет ли заданный тип определённому интерфейсу или условию. Другими словами, он гарантирует, что у типа есть все необходимые свойства и методы определённого интерфейса. Это способ убедиться, что переменная соответствует определению типа. Пример:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Аннотация типа с использованием `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// В следующих строках TypeScript не сможет правильно вывести типы
user.attributes?.map(console.log);   // Свойство 'map' не существует в типе 'string | string[]'. Свойство 'map' не существует в типе 'string'.
user.nickName; // string | string[] | undefined

// Утверждение типа с использованием `as`
const user2 = {
```

```
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Здесь TypeScript тоже не сможет правильно вывести типы
user2.attributes?.map(console.log);   // Свойство 'map' не существует в типе 'string | string[]'. Свойство 'map' не существует в типе 'string'.
user2.nickName; // string | string[] | undefined

// С оператором `satisfies` теперь можно правильно вывести типы
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);   // TypeScript правильно выводит тип: string[]
user3.nickName; // TypeScript правильно выводит тип: undefined
```

Импорт и экспорт только типов

Импорт и экспорт только типов позволяют импортировать или экспортировать типы, не импортируя и не экспортируя связанные с ними значения или функции. Это может быть полезно для уменьшения размера сборки.

Для импорта только типов можно использовать конструкцию `import type`.

TypeScript разрешает использовать в импорте только типов расширения как файлов объявлений, так и файлов реализации (.ts, .mts, .cts и .tsx) независимо от настроек `allowImportingTsExtensions`.

Пример:

```
import type { House } from './house.ts';
```

Поддерживаются следующие формы:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

Объявление using и явное управление ресурсами

Объявление `using` — это неизменяемая привязка с блочной областью видимости, подобная `const`, которая используется для управления освобождаемыми ресурсами. При инициализации значением метод `Symbol.dispose` этого значения регистрируется, а затем выполняется при выходе из охватывающей блочной области видимости.

Эта возможность основана на управлении ресурсами в ECMAScript, которое полезно для выполнения необходимых задач очистки после создания объекта, таких как закрытие соединений, удаление файлов и освобождение памяти.

Примечания:

- Поскольку эта возможность появилась только в TypeScript версии 5.2, большинство сред выполнения не поддерживают её нативно. Вам понадобятся полифилы для: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.

- Кроме того, потребуется настроить tsconfig.json следующим образом:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Пример:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Простой полифил

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Ресурс объявлен
    console.log(2);
}     //     Ресурс     освобождён     (например,     выполняется
`work[Symbol.dispose]()`)

console.log(3);
```

Код выведет:

```
1
2
```

```
disposed
3
```

Ресурс, который можно освободить, должен соответствовать интерфейсу `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Объявления `using` записывают операции освобождения ресурсов в стек, гарантируя их освобождение в порядке, обратном порядку объявления:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // освобождает `C`, затем `B`, затем `A`.
```

Освобождение ресурсов гарантируется даже при выполнении последующего кода или возникновении исключений. При этом операция освобождения потенциально может выбросить исключение и, возможно, подавить другое. Для сохранения информации о подавленных ошибках введено новое встроенное исключение `SuppressedError`.

Объявление await using

Объявление `await using` обрабатывает асинхронно освобождаемый ресурс. Значение должно иметь метод `Symbol.asyncDispose`, выполнение которого будет ожидаться в конце блока.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Ресурс объявлен
}   //   Ресурс   освобождён   (например,   выполняется   `await
work[Symbol.asyncDispose]()`)
```

Асинхронно освобождаемый ресурс должен соответствовать интерфейсу `Disposable` или `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Простой полифил

class DatabaseConnection implements AsyncDisposable {
    // Метод, вызываемый при асинхронном освобождении объекта
    [Symbol.asyncDispose]() {
        // Закрыть соединение и вернуть промис
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
             await  new  Promise(resolve  =>  setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
      // Создать соединение и асинхронно освободить его при
выходе из области видимости
      await using connection = new DatabaseConnection(); //
Ресурс объявлен
    console.log('Doing some work...');
```

```
} // Ресурс освобождён (например, выполняется `await connection[Symbol.asyncDispose]()`)

doWork();
```

Код выводит:

```
Doing some work...
Closing the connection...
Connection closed.
```

Объявления `using` и `await using` разрешены в инструкциях: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Атрибуты импорта

Атрибуты импорта в TypeScript 5.3 (метки для импортов) сообщают среде выполнения, как обрабатывать модули (JSON и т. д.). Это повышает безопасность, обеспечивая явный импорт, и согласуется с политикой безопасности контента (CSP), делая загрузку ресурсов безопаснее. TypeScript проверяет их допустимость, но интерпретацию для обработки конкретных модулей оставляет среде выполнения.

Пример:

```
import config from './config.json' with { type: 'json' };
```

с динамическим импортом:

```
const config = import('./config.json', { with: { type: 'json' } });
```

Проверка синтаксиса регулярных выражений

Начиная с TypeScript 5.5.4, на этапе компиляции выполняется проверка литералов регулярных выражений на распространённые ошибки (например, недопустимый синтаксис, неверные обратные ссылки и возможности, не поддерживаемые целевой версией JS). Это помогает раньше обнаруживать ошибки, но строки `new RegExp("...")` не проверяются.

```
let r = /(a)\2/; // Ошибка: эта обратная ссылка ссылается на несуществующую группу.
```

import defer

`import defer` позволяет загрузить модуль, но отложить его выполнение до фактического использования чего-либо из него. Это помогает избежать ненужной работы и побочных эффектов.

- Работает только с: `import defer * as name from "module"`
- Код выполняется только при обращении к экспортируемому значению